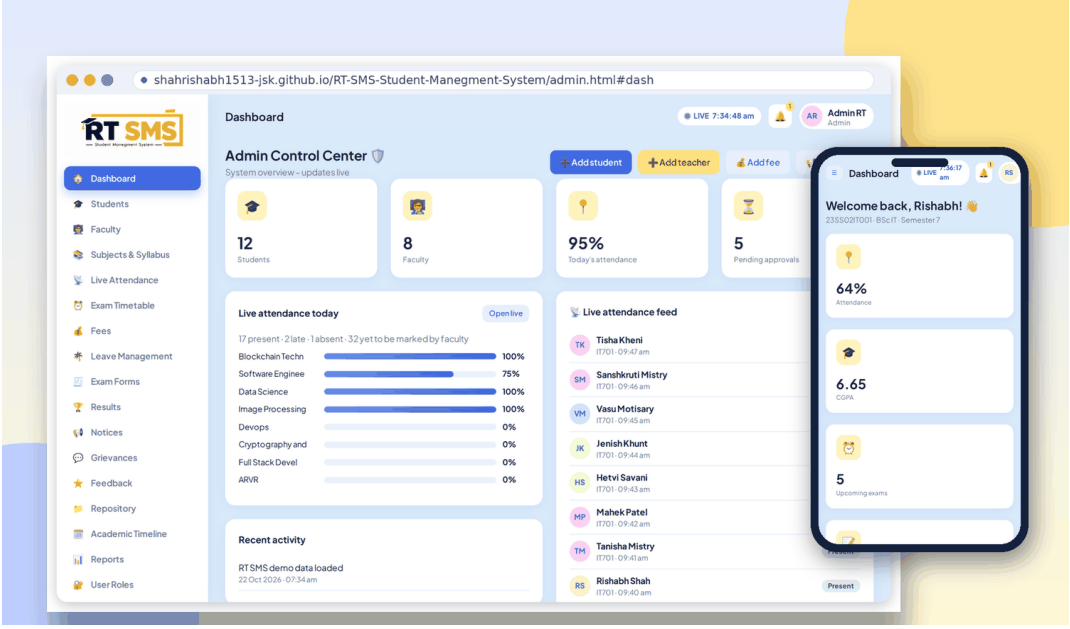
A PERSONAL PROJECT REPORT

RT SMS

— Student Managment System —

STUDENT MANAGEMENT SYSTEM

A Role-Based Web Portal for Admin, Faculty and Student using HTML, CSS and JavaScript



3	19	28	0
ROLE PORTALS	ADMIN MODULES	SUBJECTS · 8 SEMESTERS	BACKEND · NO DATABASE

Project Title	**RT SMS – Student Management System**
Technologies	HTML5 · CSS3 · JavaScript (ES6+) · LocalStorage · jsPDF · QR code
Project Type	Personal project (front-end only, no backend)
Developer ID	RT1504
Live Project	Live Project
Source Code	Project Repository
GitHub	github.com/shahrishabh1513-jsk
LinkedIn	linkedin.com/in/rishabh-alpeshabhai-shah

DESIGNED AND DEVELOPED BY

SHAH RISHABH ALPESHBHAI

PROJECT AT A GLANCE one page summary

RT SMS is a complete campus management web portal. Admin, Faculty and Student log in to their own portal, and every action — attendance, leave, fees, exam forms, marks and results — updates the other portals **live**, using only HTML, CSS and JavaScript and the browser's storage.

ADMIN PORTAL
- Manage students & faculty
- Fees, roles, settings
- Generate & publish results
- Reports, CSV, JSON backup

FACULTY PORTAL
- Take live attendance
- Approve leave requests
- Verify paid exam forms
- Enter internal + external marks

STUDENT PORTAL
- Attendance, timetable, notes
- Exam form + online payment
- Hall ticket & result PDFs
- Leave, grievance, feedback

LIVE SYNC
Tabs update instantly through BroadcastChannel

ONLINE PAYMENTS
UPI QR, card and net-banking demo gateway

PDF DOCUMENTS
Marksheet, hall ticket, syllabus, notes

SMART RESULTS
SGPA, CGPA, grade points and class rank

RESPONSIVE UI
Desktop, tablet and mobile ready

35 / 35 TESTS
Automated browser tests all passed

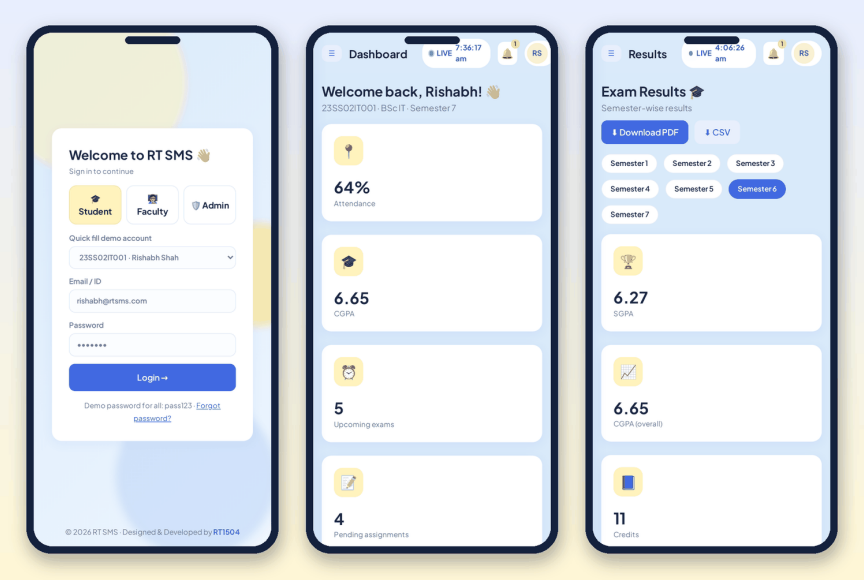
HOW IT WORKS

1. **Login by role**
2. **Use your portal**
3. **Action is saved**
4. **Live sync to all tabs**
5. **Results · PDF · reports**

HTML5 | CSS3 | JavaScript ES6+ | localStorage | Broadcast API | jsPDF | QR code | GitHub Pages

DEMO LOGINS (password for all: pass123)
Admin: admin@rtsms.com · RTSMSAD001
Faculty: neha@rtsms.com · RTSMSFE001–008
Student: rishabh@rtsms.com · 23SS02IT001–012

QUICK LINKS
Live: Live Project
Code: Project Repository
GitHub: GitHub Profile
LinkedIn: LinkedIn Profile



Fully responsive – works on phone, tablet and desktop

CONTENTS

Section	Title	Page
	Certificate	1
	Acknowledgement	2
	Abstract	3
CHAPTER 1	INTRODUCTION	4
1.1	Project Overview	4
1.2	Problem Statement	4
1.3	Objectives of the Project	4
1.4	Scope of the Project	5
1.5	Users and Roles	5
CHAPTER 2	TOOLS AND TECHNOLOGIES	6
2.1	HTML5	6
2.2	CSS3	6
2.3	JavaScript (ES6+)	6
2.4	Browser Storage and Live Sync	6
2.5	External Libraries	7
2.6	Development and Deployment Tools	7
CHAPTER 3	PROJECT DESIGN AND MODULES	8
3.1	System Architecture	8
3.2	Authentication and Role-Based Routing	9
3.3	Admin Portal Modules	9
3.4	Faculty Portal Modules	10
3.5	Student Portal Modules	10
3.6	Data Model	11
3.7	Live Synchronisation	11
3.8	Core Workflows	12
CHAPTER 4	SYSTEM IMPLEMENTATION	21
4.1	Project Structure	21
4.2	Application Shell and Router	21
4.3	Important Functions	21
4.4	Code Samples	22
4.5	PDF and QR Implementation	22
4.6	Development and Deployment	23
4.7	Security and Validation Notes	23
CHAPTER 5	SOFTWARE TESTING	24
5.1	Authentication and Access Control	24
5.2	Attendance and Leave	25
5.3	Exam Form, Fees and Results	25
5.4	Notices, Grievance, Admin and Responsive UI	25
5.5	Test Case Summary	26
CHAPTER 6	SCREENSHOTS AND WALKTHROUGH	27
6.1	Login Page	27
6.2	Admin Portal	27
6.3	Faculty Portal	34
6.4	Student Portal	37
6.5	Generated PDF Documents	43
6.6	Mobile View	43
CHAPTER 7	LEARNING OUTCOMES	44
7.1	Skills and Knowledge Gained	44
7.2	Challenges and Solutions	44
7.3	Limitations	44
CHAPTER 8	CONCLUSION	45
8.1	Conclusion	45

Section	Title	Page
8.2	Future Enhancements	45
CHAPTER 9	REFERENCES AND GLOSSARY	46
9.1	References	46
9.2	Glossary	46

CERTIFICATE

This is to certify that **SHAH RISHABH ALPESHBHAI** has successfully completed the personal project entitled **"RT SMS – Student Management System: A Role-Based Web Portal Using HTML, CSS and JavaScript"**.

The project was designed, developed, tested and documented by the candidate using HTML5, CSS3 and Vanilla JavaScript. The work presented in this report is original and has been carried out sincerely and to the best of the candidate's ability.

This report is submitted as a record of the candidate's personal project work.

_____

Project Guide
(Name & Signature)

Date: _____

_____

Head of Department
(Name & Signature)

Place: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the planning, development and documentation of my personal project, **RT SMS – Student Management System**.

I am thankful to my project guide and the faculty members of my department for their valuable guidance, encouragement and constructive suggestions, which helped me improve the quality of this work. Their feedback motivated me to focus on clean code, a clear workflow for every user role and a better user experience.

I also thank my family and friends for their constant support, patience and motivation throughout the project. I am grateful to the open web community and to the creators of freely available learning resources such as MDN Web Docs, and of the open-source libraries jsPDF and qrcode-generator, which made it possible to learn and apply modern web technologies effectively.

Finally, I thank everyone who directly or indirectly helped me in completing this project successfully.

SHAH RISHABH ALPESHBHAI

ABSTRACT

RT SMS (Student Management System) is a web-based campus management portal that brings the daily work of an institute into one modern and responsive website. It is developed using only HTML5, CSS3 and Vanilla JavaScript (ES6+), without any framework, backend server or database. The complete system runs in a modern browser and is also hosted as a live website on GitHub Pages.

The system provides three connected portals — **Admin**, **Faculty** and **Student** — protected by a role-based login. Admin manages students, faculty, subjects, fees, roles, notices, reports and result publishing. Faculty take attendance, approve leave, verify exam forms and enter internal and external marks. Students view attendance, apply for leave, fill the exam form and pay fees online, raise grievances, give feedback, submit assignments and download results.

All data is stored in the browser using LocalStorage and is synchronised live between open tabs through the BroadcastChannel API and the storage event, so an action in one portal appears instantly in the others. Marksheets, hall tickets, syllabus books and notes are generated as PDF files with jsPDF, and fee payment supports a UPI QR code, card and net-banking demonstration gateway.

The project contains 8 semesters and 28 subjects of demo data, a grade-point and SGPA/CGPA engine, and a responsive light-theme interface. A set of 35 automated browser tests covering login, attendance, leave, payments, results, notices, administration and responsive layout all passed. The project demonstrates DOM manipulation, state management, event-driven design, document generation and workflow design.

Keywords: *Student Management System, HTML5, CSS3, JavaScript, LocalStorage, BroadcastChannel, jsPDF, Role-Based Access, Responsive Design*

CHAPTER 1

INTRODUCTION

1.1 Project Overview

RT SMS (Student Management System) is a browser-based campus portal that connects three kinds of users — the **Administrator**, the **Faculty** and the **Student** — in one website. Instead of keeping attendance in registers, leave in paper forms and results in spreadsheets, every task is done online and every portal sees the same up-to-date data.

The project is written only with **HTML5, CSS3 and Vanilla JavaScript**. There is no server and no database: all records are stored in the browser's LocalStorage, and open tabs are synchronised live with the BroadcastChannel API. The website is published on GitHub Pages, so it can be opened from any device with a modern browser.

Main highlights of the project:

- Role-based login for Admin, Faculty and Student with separate dashboards and menus.
- Live attendance (Present / Late / Absent) with a configurable minimum-attendance alert.
- Leave management with four leave types and yearly balance.
- Exam form, online fee payment (UPI QR, card, net banking) and downloadable hall ticket.
- Marks entry (internal /30 + external /70), grade points, SGPA, CGPA and class rank.
- Notices, grievances, faculty feedback, assignments, notes and an academic timeline.
- PDF generation for result marksheets, hall tickets, syllabus book, notes and assignments.

AT A GLANCE

3 portals · 19 admin modules · 28 subjects across 8 semesters · 12 demo students · 8 demo faculty · 35 automated tests passed.

1.2 Problem Statement

Many colleges and training institutes still manage attendance, leave, fees and results using separate registers, messages and spreadsheets. This causes delay, duplicate work and mistakes: a student cannot see his or her attendance until the register is updated, a leave approval has to be chased by hand, and results reach students only after long manual calculation.

Large commercial ERP systems solve these problems but are expensive and heavy for small institutes and for learning. There is a need for a **simple, free and fast** system that runs in the browser, shows live information to every role, and can also be used as a hands-on learning project for web development.

1.3 Objectives of the Project

- To build one connected portal for Admin, Faculty and Student using only front-end web technologies.
- To keep attendance, leave, fees, exam forms and results in one consistent data model.

- To make every change visible to the other roles instantly, without reloading the page.
- To automate SGPA / CGPA calculation, grading and class ranking.
- To generate official-looking PDF documents directly in the browser.
- To provide a responsive, clean interface that works on desktop, tablet and mobile.
- To test the system with automated browser tests and document the results.

1.4 Scope of the Project

Included in the scope: role-based authentication, user management, subjects and syllabus for 8 semesters, attendance, leave, fees and exam-form payment (demonstration gateway), marks and results, notices, grievance, feedback, repository (notes and assignments), reports, CSV / JSON export and PDF generation.

Not included: a real server or database, real payment gateway integration, password hashing, and e-mail / SMS delivery. These are listed as future enhancements in Chapter 8.

1.5 Users and Roles

Role	Main responsibilities	Typical actions
Admin	Controls the whole system and publishes results.	Add students / faculty, add fees, manage roles, post notices, publish results, view reports, backup data.
Faculty	Teaches subjects and records academic data.	Take attendance, approve leave, verify exam forms, enter marks, create assignments, reply to grievances.
Student	Uses the services and tracks own progress.	View attendance and results, apply leave, fill exam form and pay, give feedback, download hall ticket.

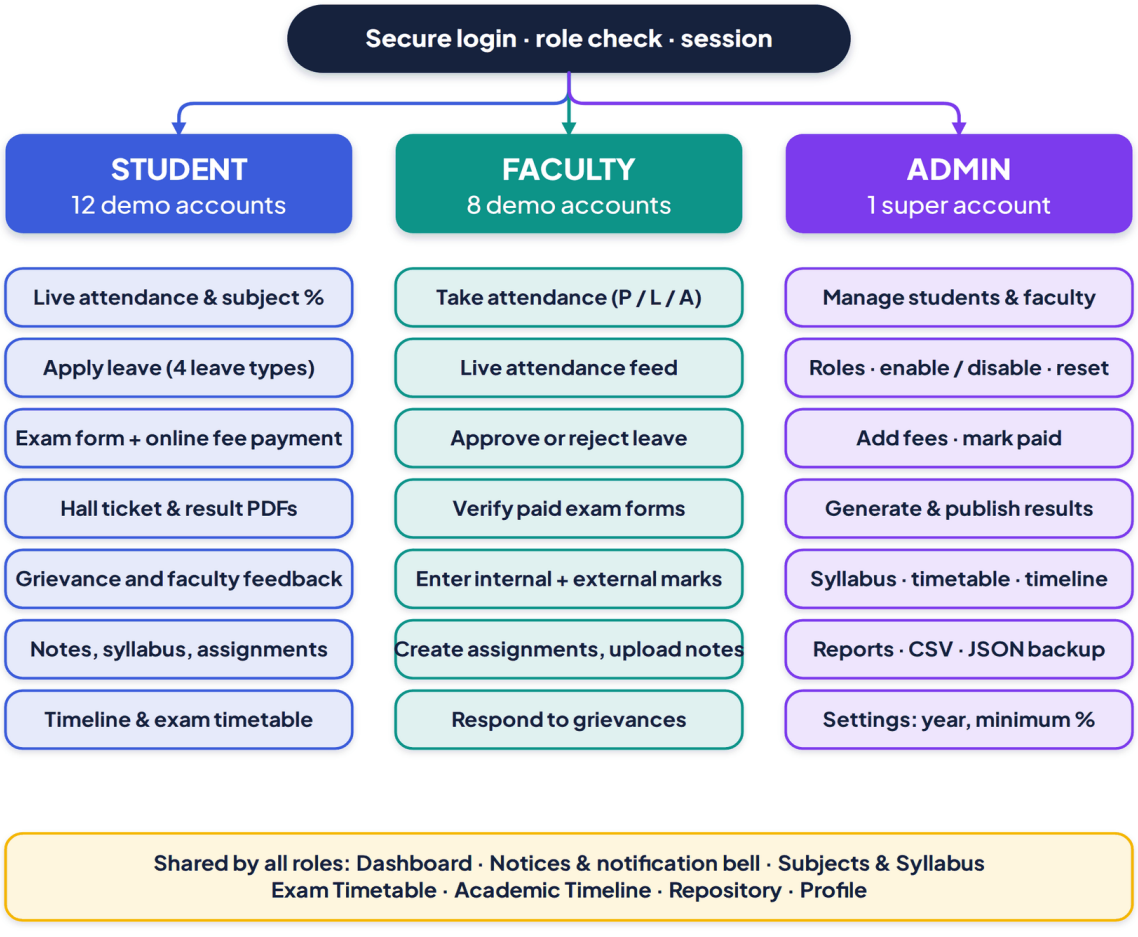


Figure 1.1: Role capability map of RT SMS

CHAPTER

2

TOOLS AND TECHNOLOGIES

2.1 HTML5

HTML5 gives the structure of every page. The project has four entry pages — `index.html` (login), `student.html`, `teacher.html` and `admin.html` — each containing only a small shell. The sidebar, dashboards, tables and forms are created by JavaScript from templates, which keeps the files short and consistent.

- **Semantic structure** for the sidebar, top bar, main content and footer.
- **Form controls** such as text, number, date, select, checkbox and file inputs.
- **Dialogs** for add / edit forms, payment and notification lists.
- **Accessibility attributes** such as labels, titles and keyboard-friendly buttons.

2.2 CSS3

One shared stylesheet, `assets/css/style.css`, controls the colours, layout and responsive behaviour of all portals. The look is a clean light theme: royal-blue primary colour, yellow accent taken from the logo, white cards and soft shadows, with the **Plus Jakarta Sans** font.

- **CSS variables** for colours, so the whole theme can change in one place.
- **Flexbox and Grid** for the sidebar layout, stat cards and dashboards.
- **Media queries** (about 860 px) turn the sidebar into a slide-in mobile menu.
- **Transitions and animations** for counters, progress bars and toast messages.
- **Print rules** that hide the sidebar when a page is printed.

2.3 JavaScript (ES6+)

JavaScript is the brain of RT SMS. It performs login, routing between pages, data storage, calculations, live synchronisation and PDF creation. The code is split into small files so that each role and each concern has its own place.

- **DOM manipulation:** every page is rendered with template literals and inserted into the main container.
- **State object `S`:** one JavaScript object that holds all 19 collections (users, courses, attendance, marks and so on).
- **Array methods:** `map`, `filter`, `find`, `reduce` and `sort` are used for tables, reports and ranking.
- **Event handling:** buttons call functions that change the state and then call `sv()` to save and `render()` to redraw.
- **Algorithms:** grade lookup, SGPA / CGPA, class rank, fee calculation and attendance percentage.

2.4 Browser Storage and Live Sync

RT SMS needs no server because the browser itself stores the data.

API	Used for	Key / detail
localStorage	All application data (users, attendance, marks, fees …)	Key `rtsms3`, data version 3
sessionStorage	The logged-in user of the current tab	Key `rtme`
BroadcastChannel	Tells other tabs that data has changed	Channel `rtsms3`
storage event	Fallback notification between tabs	Triggers `sync()` in other tabs

2.5 External Libraries

Library	Version	Purpose
jsPDF	2.5.1	Creates PDF files (marksheet, hall ticket, syllabus, notes) in the browser
qrcode-generator	1.4.4	Draws the UPI payment QR code
Google Fonts	—	Plus Jakarta Sans typeface

If the internet is not available, the application still works; only PDF download and the QR code need the libraries, and the app shows a friendly message in that case.

2.6 Development and Deployment Tools

Tool	Details
Languages	HTML5, CSS3, JavaScript (ES6+)
Code editor	Visual Studio Code with Live Server preview
Version control	Git and GitHub (repository RT-SMS-Student-Manegment-System)
Hosting	GitHub Pages (static hosting, no build step)
Browsers tested	Chromium (automated), responsive modes 390 / 768 / 1366 px
Testing tool	Playwright browser automation

CHAPTER

3

PROJECT DESIGN AND MODULES

3.1 System Architecture

RT SMS is a client-side application: everything runs in the user's browser. The code is arranged in four layers. The presentation layer shows pages; the application layer holds the logic of each role; the core engine provides shared functions; and the data layer stores and synchronises the state.

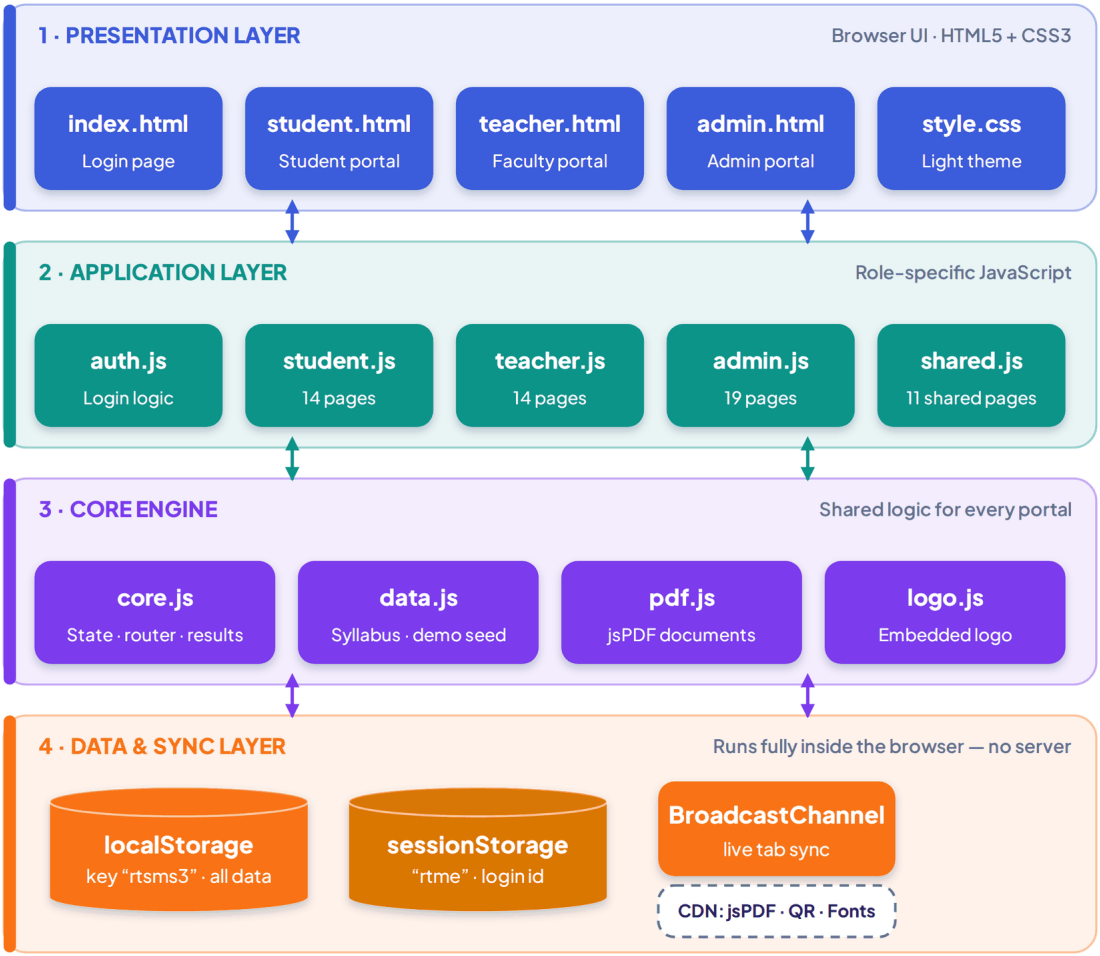


Figure 3.1: Layered architecture of RT SMS

File	Responsibility
index.html + auth.js	Login page, role tabs, demo-account quick fill, session creation and redirect.
admin.js / teacher.js / student.js	Menu list and page functions that are specific to one role.

File	Responsibility
shared.js	Pages used by several roles: notices, leave, grievance, feedback, repository, syllabus, timetable, timeline, profile.
core.js	State object, save / load, router, shell (sidebar + top bar), helpers, grade and SGPA engine, sync.
data.js	Syllabus of 8 semesters and the seed data for demo users, attendance, marks, fees and notices.
pdf.js	All jsPDF functions: header, key-value grid, table, footer, hall ticket, result, syllabus, notes.
style.css	Complete light theme and responsive rules.

3.2 Authentication and Role-Based Routing

A user chooses a role tab, enters the ID or e-mail and the password, and presses Login. The system checks that the account exists **for that role**, that the password matches and that the account is active. On success the user id is stored in `sessionStorage` and the browser is redirected to the portal of that role. Every portal calls `boot(role)` first, which sends the visitor back to the login page if the session, role or status is not valid.

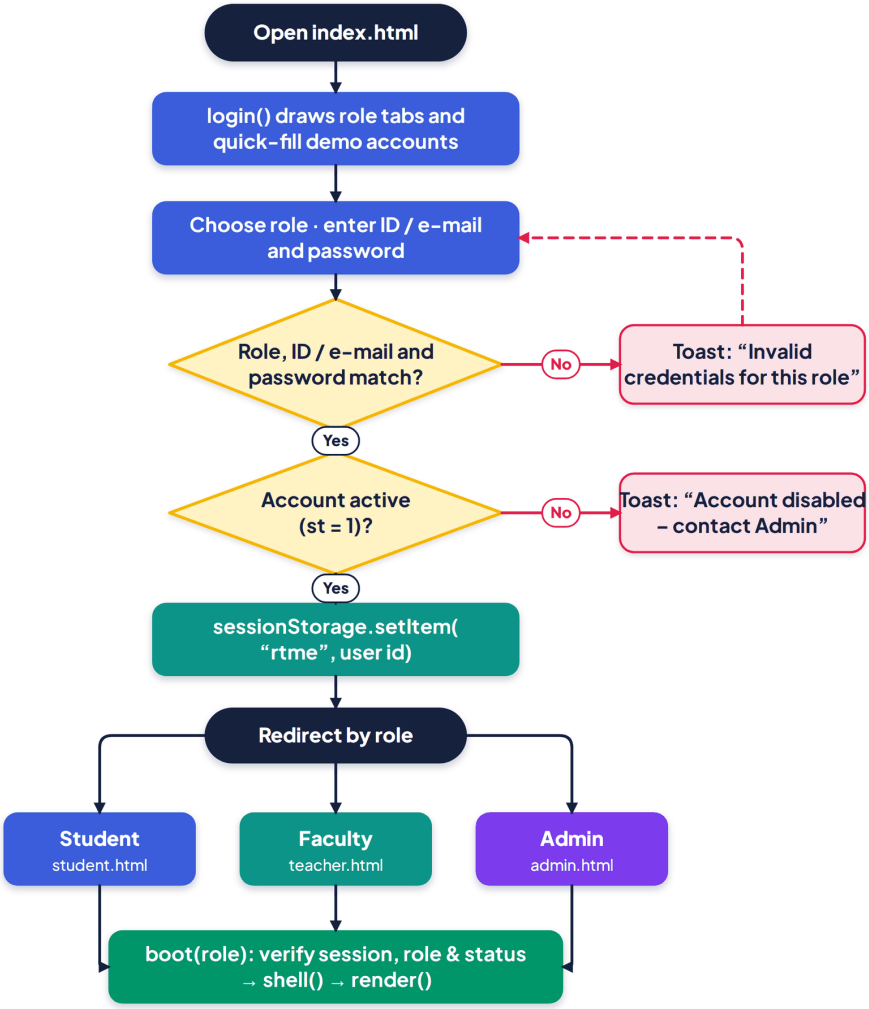


Figure 3.2: Login and role-based routing flow

3.3 Admin Portal Modules

Module	Purpose
Dashboard	Statistics, live attendance bars, live feed and recent activity
Students / Faculty	Add, edit, delete and search people; export CSV
Subjects & Syllabus	Subjects of 8 semesters with units; PDF syllabus book
Live Attendance	Real-time present / late / absent view per subject
Exam Timetable	Dates, times and rooms of exams
Fees	Add fee for one or all students, mark paid, totals
Leave Management	Review leave requests of students
Exam Forms	See submitted forms and payment status
Results	Generate and publish results per semester
Notices	Post notices by category
Grievances / Feedback	Respond to grievances; read faculty ratings
Repository	Notes, papers and assignments
Academic Timeline	Semester calendar events
Reports	Subject averages, grade distribution, low-attendance list
User Roles	Enable / disable users, reset password
Settings	Academic year, minimum attendance %, backup, restore, reset demo data

3.4 Faculty Portal Modules

Module	Purpose
Dashboard	Own subjects, today’s attendance, average rating
Take Attendance	Mark P / L / A for a subject and date
Live Attendance	Real-time class view
Results Entry	Internal /30 and external /70 marks with live grade
Leave Requests	Approve or reject student leave
Exam Verification	Verify paid exam forms
Notices, Grievances, Repository	Post notices, respond, create assignments and upload notes
Syllabus, Timetable, Timeline, Profile	Shared information pages

3.5 Student Portal Modules

Module	Purpose
Dashboard	Attendance %, CGPA, pending items, today’s attendance, weekly timetable
Attendance	Subject-wise percentage, daily record, low-attendance warning
Exam Form	Select subjects, calculate fee, pay and download hall ticket
Results	Semester marksheet, SGPA, CGPA, rank, PDF and CSV
Fees & Payments	Fee ledger, pay now, receipt
Apply Leave	Four leave types with balance and history
Grievance / Feedback	Raise issues; rate faculty

Module	Purpose
Repository	Notes, previous papers, assignment submission
Notices, Syllabus, Timetable, Timeline, Profile	Information pages and editable profile

3.6 Data Model

The complete application state is one JavaScript object `S` saved as JSON in LocalStorage. It contains 19 collections. Relationships are kept with simple ids: for example, an attendance record is stored as `S.att[date|subjectCode][studentId]` and marks as `S.mk[studentId][subjectCode] = [internal, external]`.

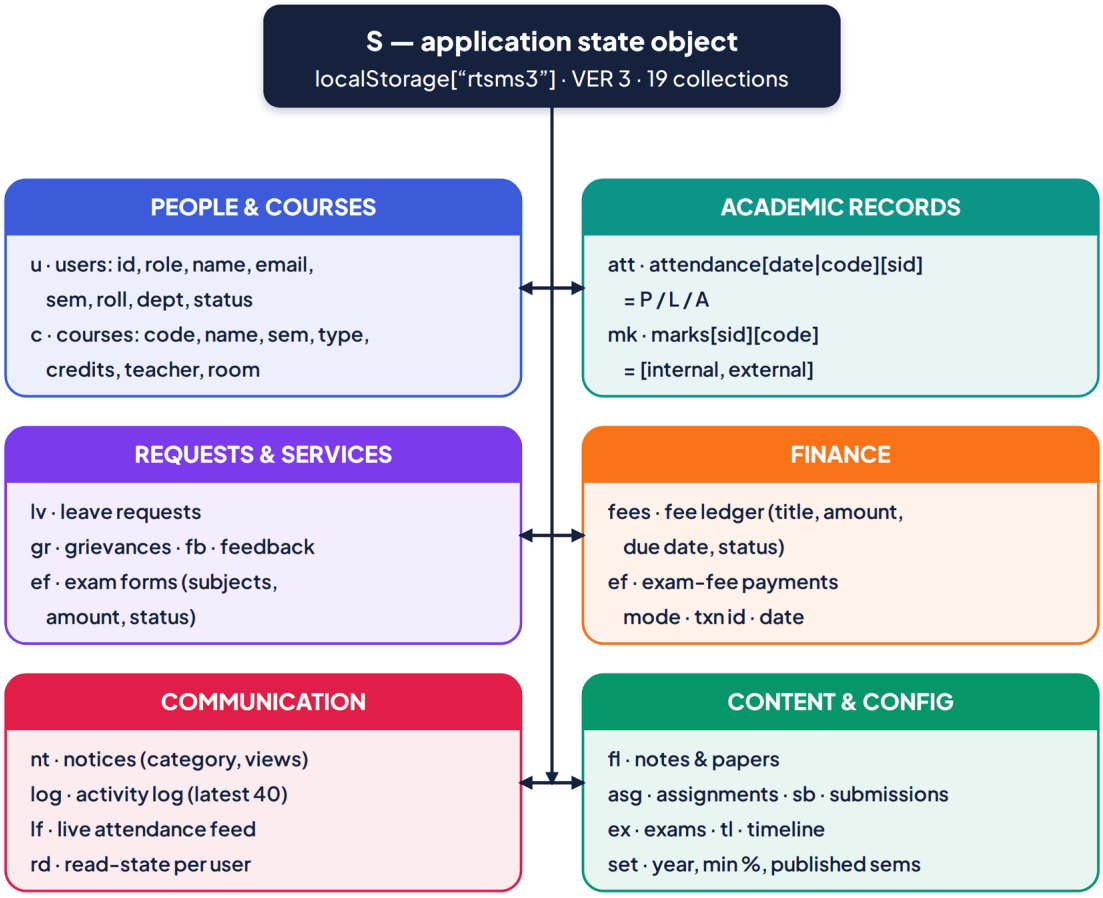


Figure 3.3: Data model of the application state

3.7 Live Synchronisation

When any portal changes the data, `sv()` writes the JSON to LocalStorage and posts a message on the BroadcastChannel. All other open tabs run `sync()`, reload the state and softly re-render the live pages (dashboard, live attendance, leave, fees and so on) — but only if no dialog is open and the user is not typing, so nobody loses input.

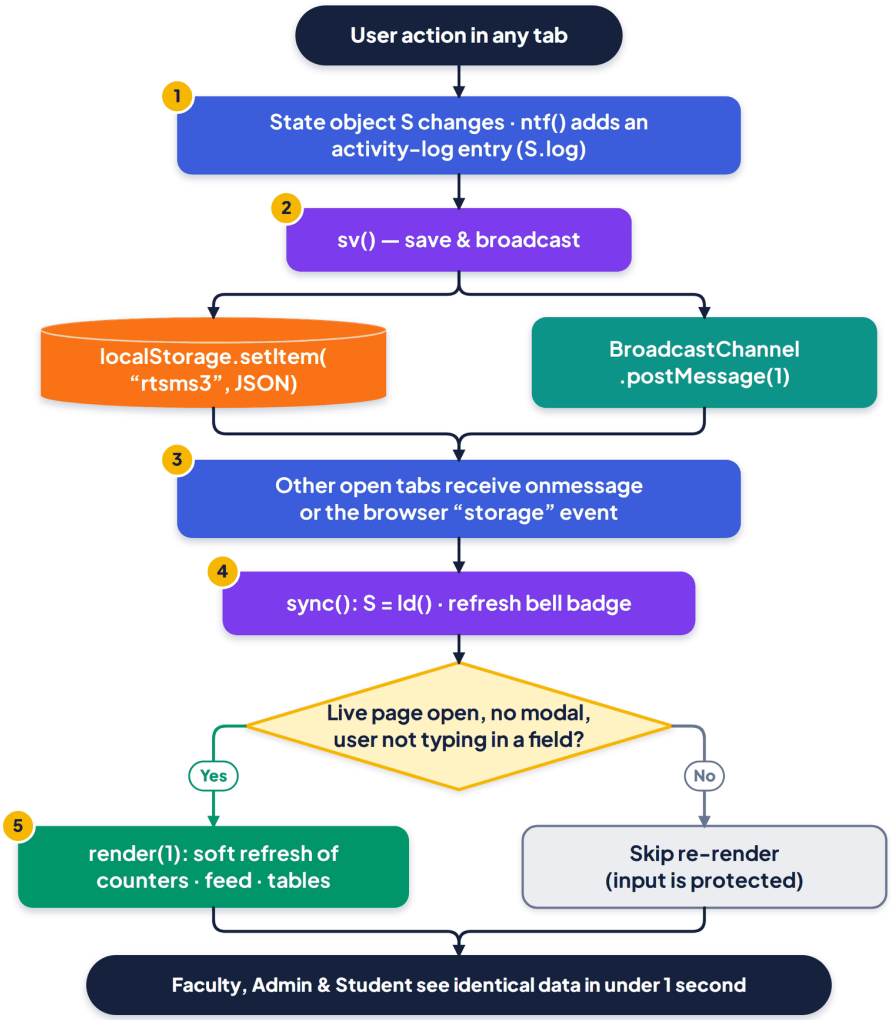


Figure 3.4: Live synchronisation between tabs

3.8 Core Workflows

The following diagrams explain the main business processes. Each one shows who acts, what the system stores and what other users see.

3.8.1 Attendance workflow

Faculty mark each student Present, Late or Absent and save. If the status of today’s record changed, a live-feed entry is created. Admin and Student screens refresh at once, and a student whose overall percentage is below the minimum (default 75 %) appears in the low-attendance report. Present and Late both count as attended.

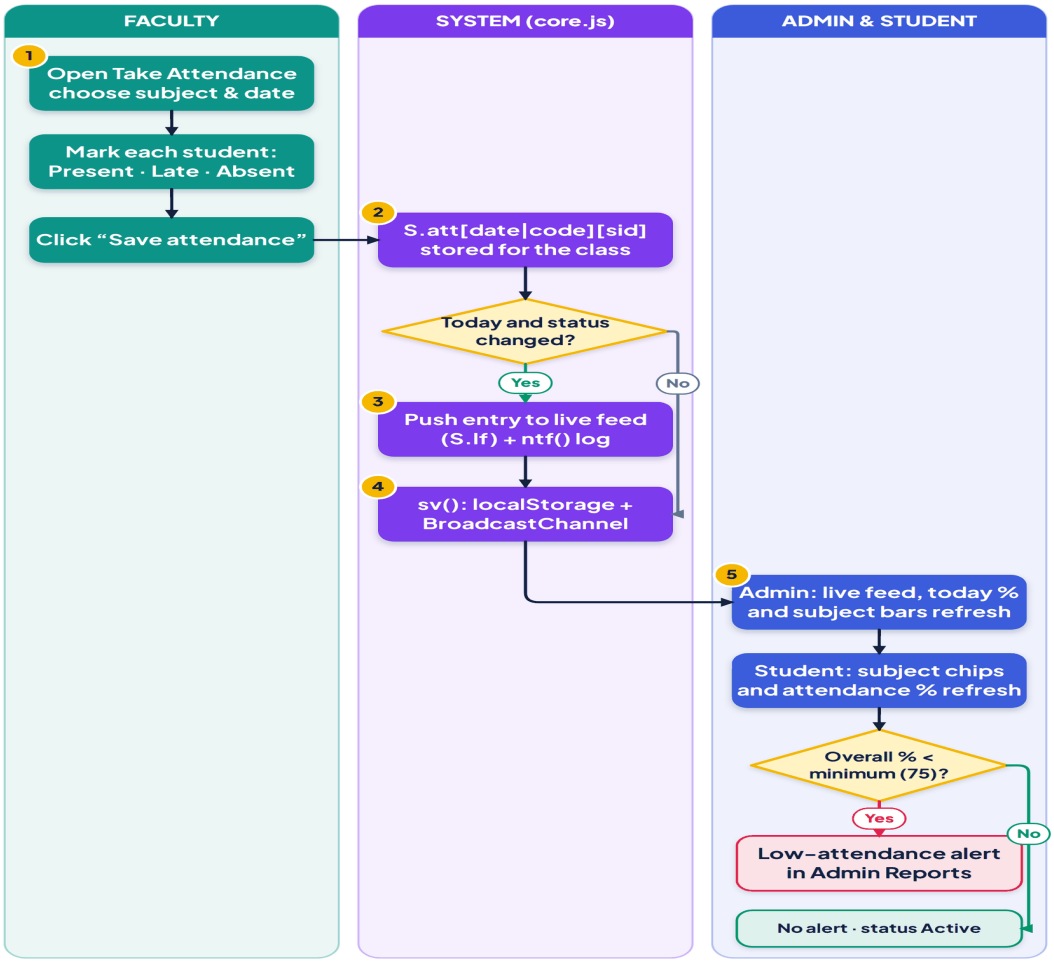


Figure 3.5: Attendance workflow

3.8.2 Leave workflow

A student chooses a leave type, dates and a reason. The request is stored as **Pending** and appears for Faculty and Admin. After approval or rejection, the student's history and remaining balance update live.

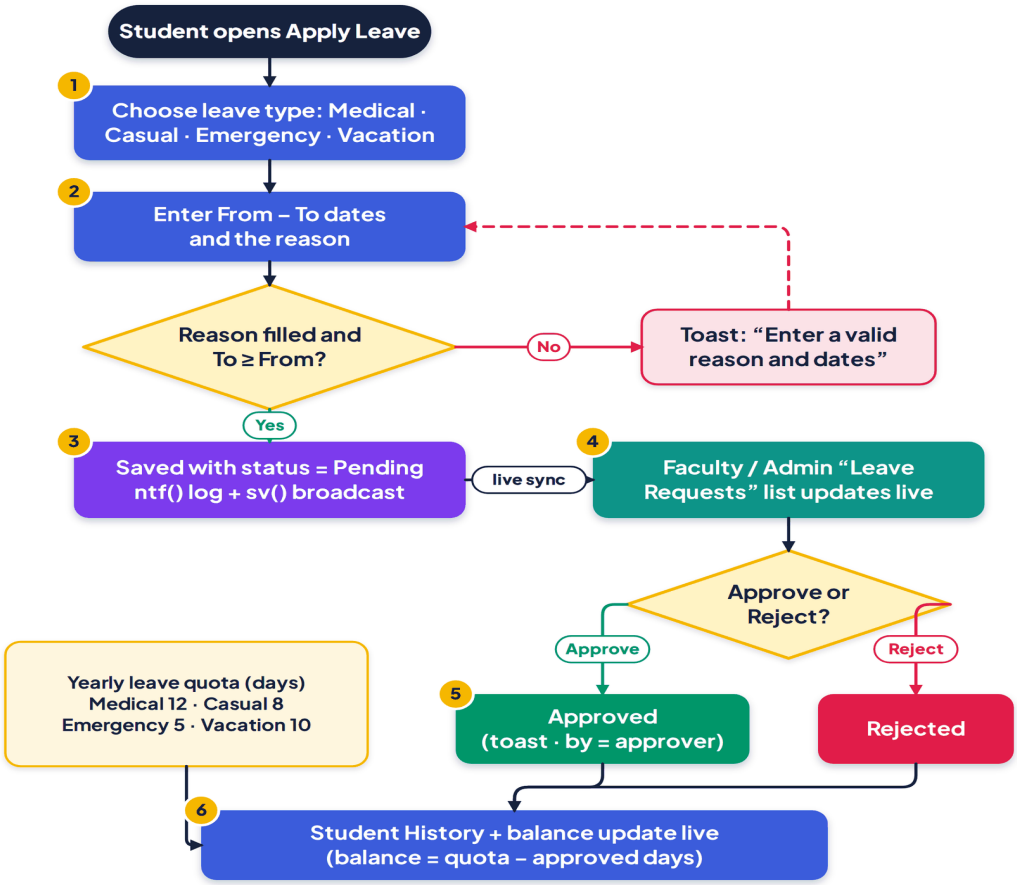


Figure 3.6: Leave management workflow

3.8.3 Exam form, payment and hall ticket workflow

The exam fee is **₹1,950 plus ₹500 for every selected subject**. After payment by UPI QR, card or net banking, the form becomes Paid and the hall ticket and receipt can be downloaded.

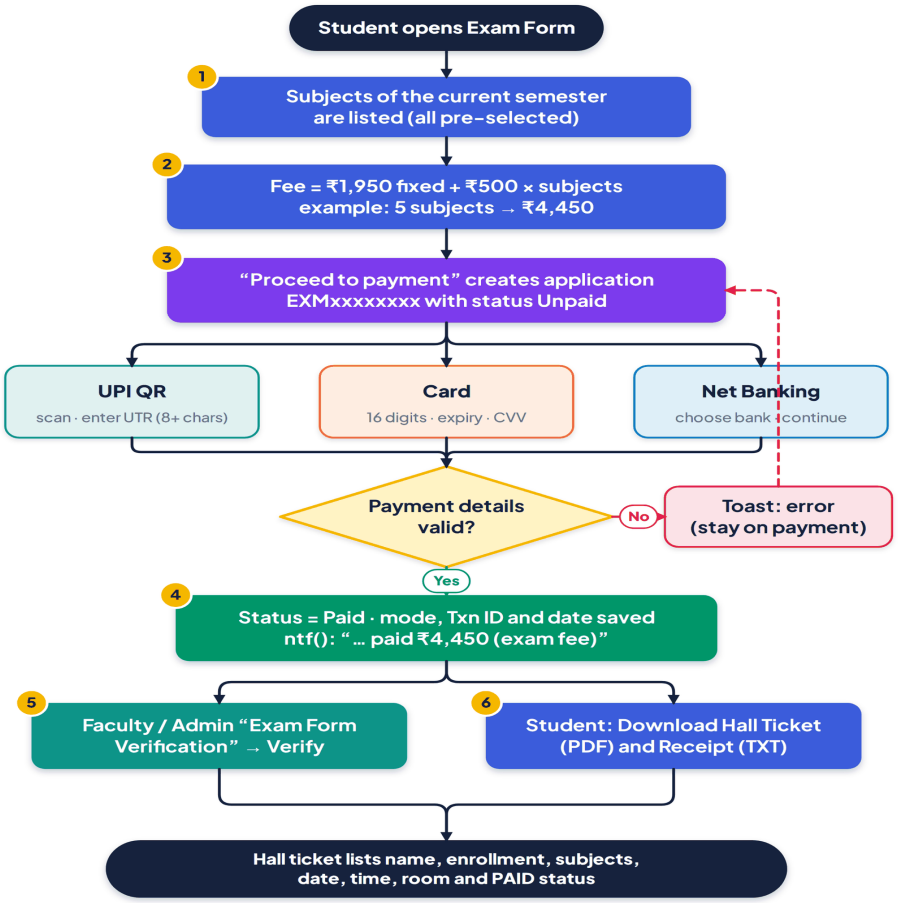


Figure 3.7: Exam form and hall ticket workflow

3.8.4 Fee management workflow

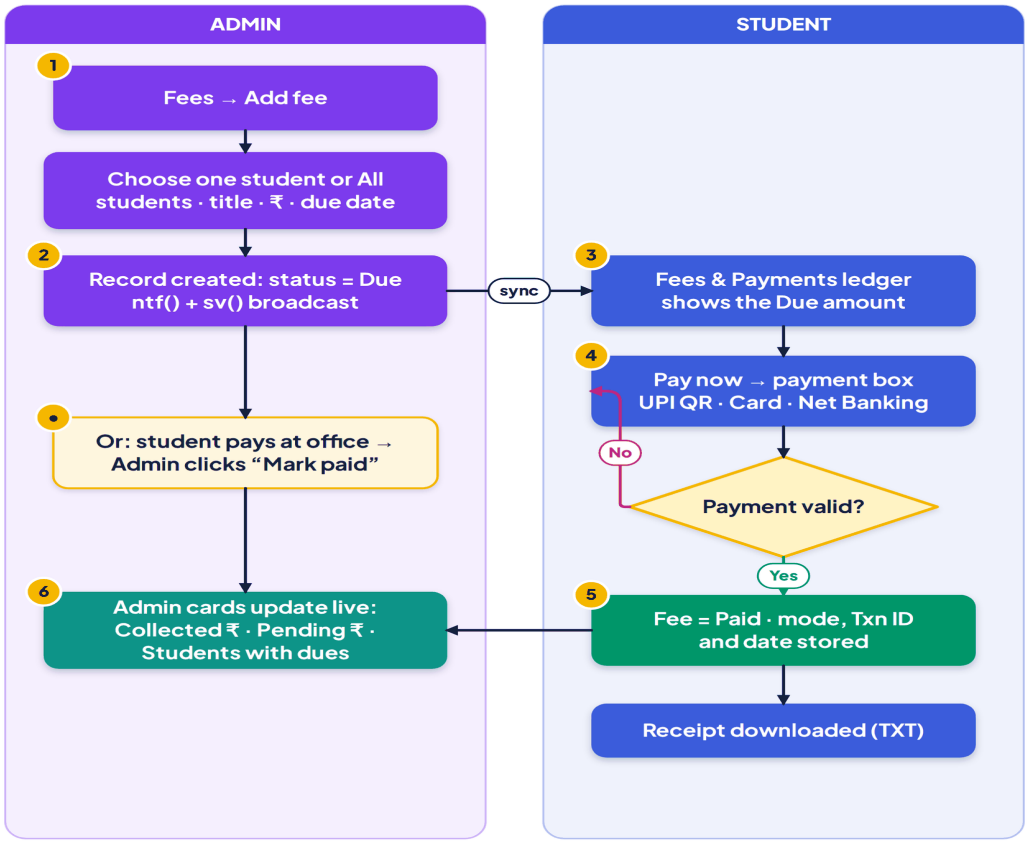


Figure 3.8: Fee management workflow

3.8.5 Marks and result workflow

Faculty enter internal (out of 30) and external (out of 70) marks. The total decides the grade. Admin publishes a semester, and students then see their marksheet with SGPA, CGPA and class rank.

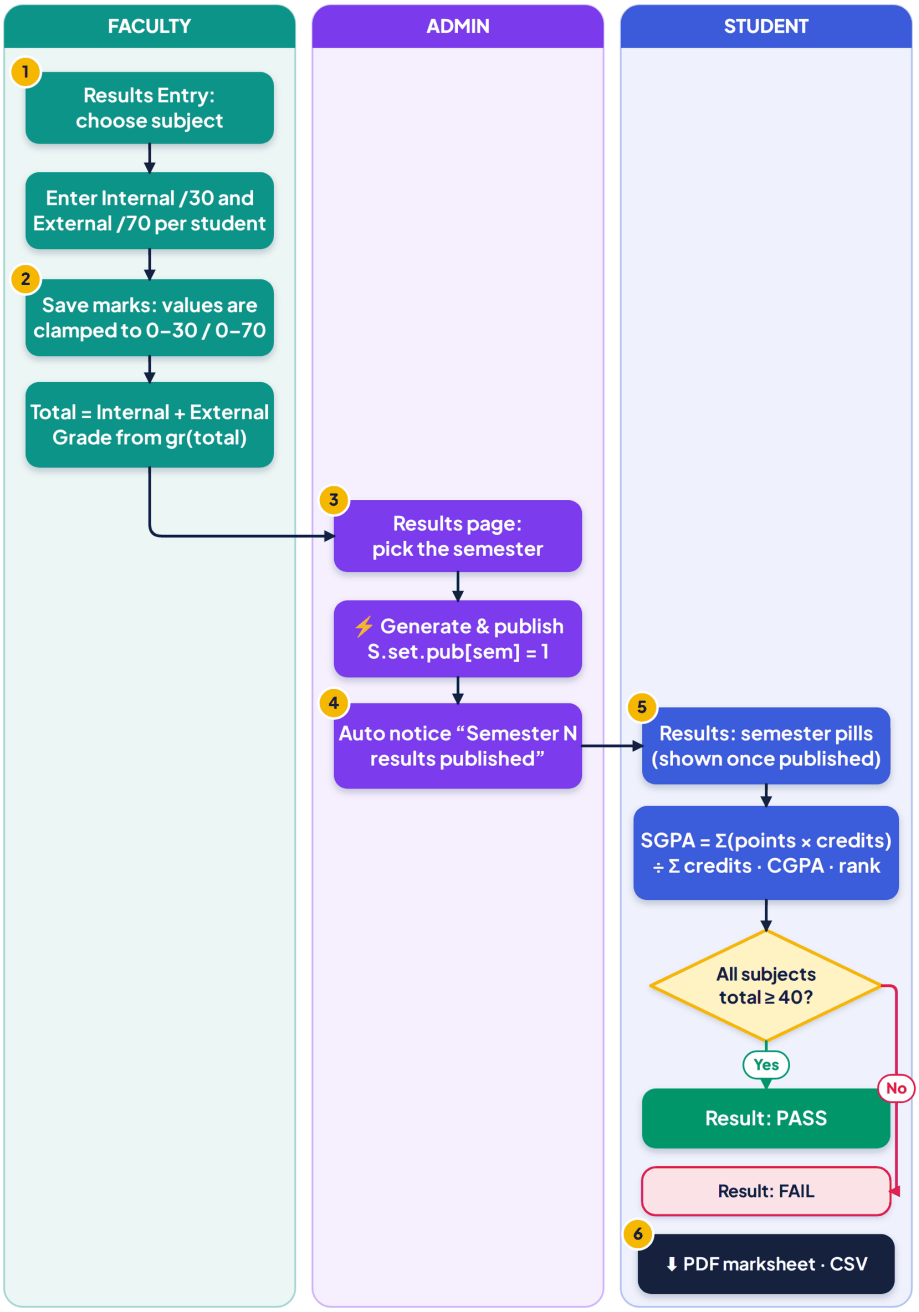


Figure 3.9: Marks to result publishing workflow

Total marks	Grade	Grade points	Total marks	Grade	Grade points
90 – 100	A+	10	50 – 59	C	6
80 – 89	A	9	40 – 49	D	5
70 – 79	B+	8	Below 40	F	0 (Fail)
60 – 69	B	7			

FORMULA

SGPA = Σ (grade points × credits) ÷ Σ credits · **CGPA** is calculated in the same way over all published semesters. · Theory subjects carry 4 credits, practicals 3.

3.8.6 Grievance and feedback workflow

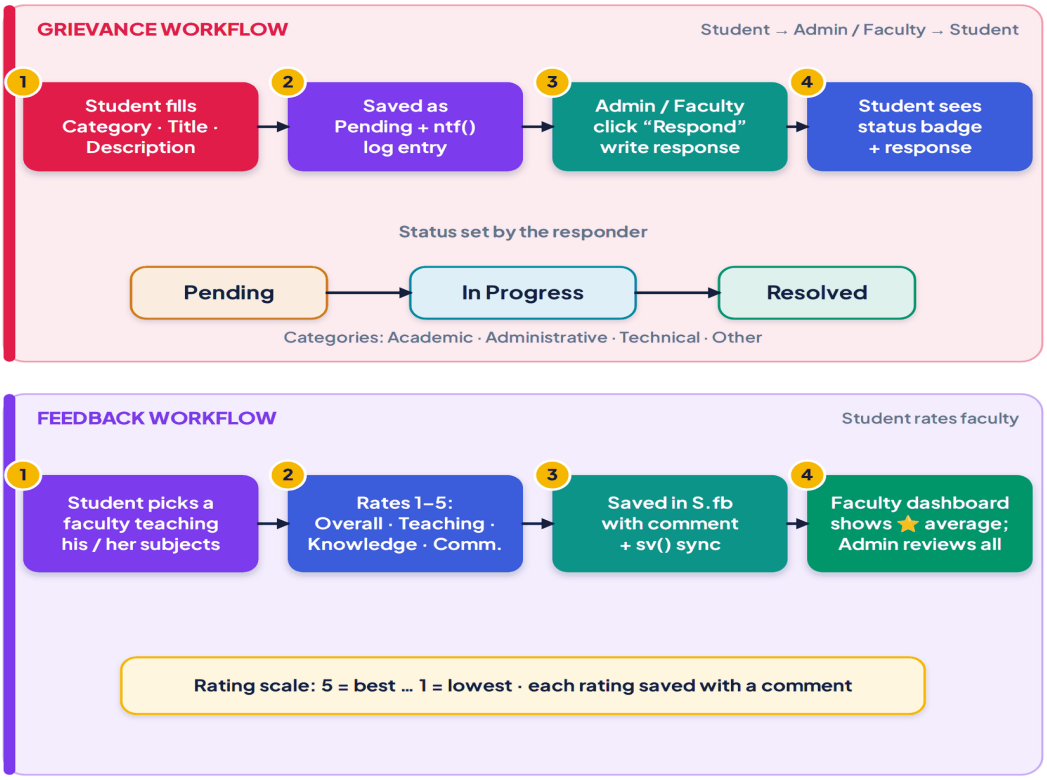


Figure 3.10: Grievance and feedback workflows

3.8.7 Assignment and repository workflow

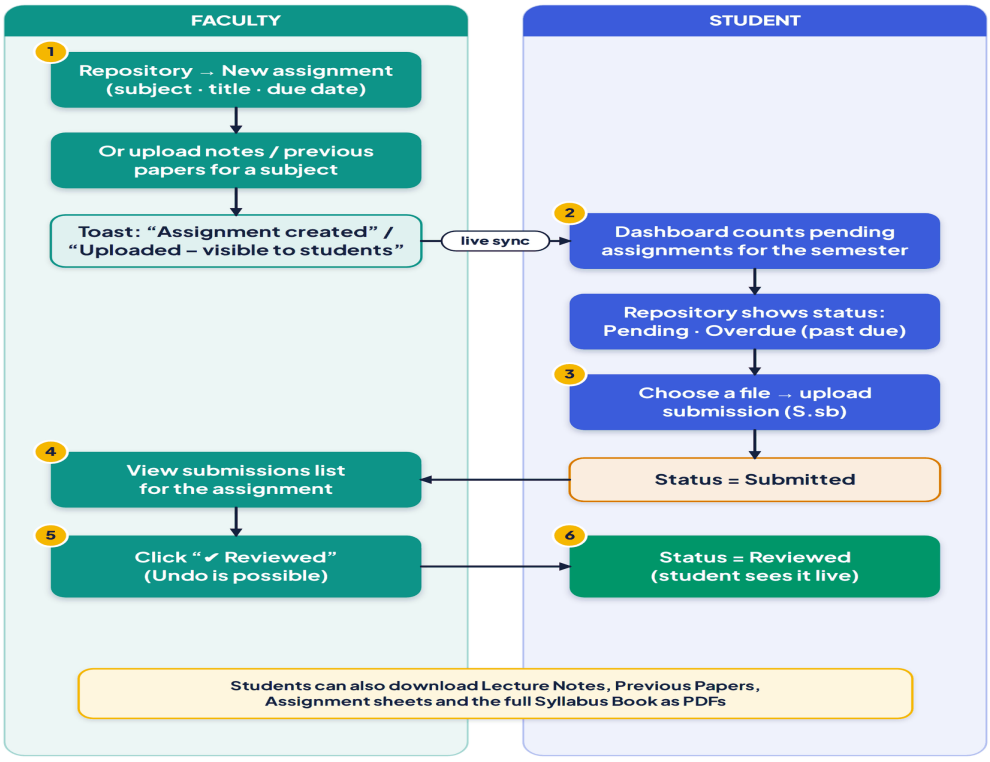


Figure 3.11: Assignment and repository workflow

3.8.8 Notification and notice workflow

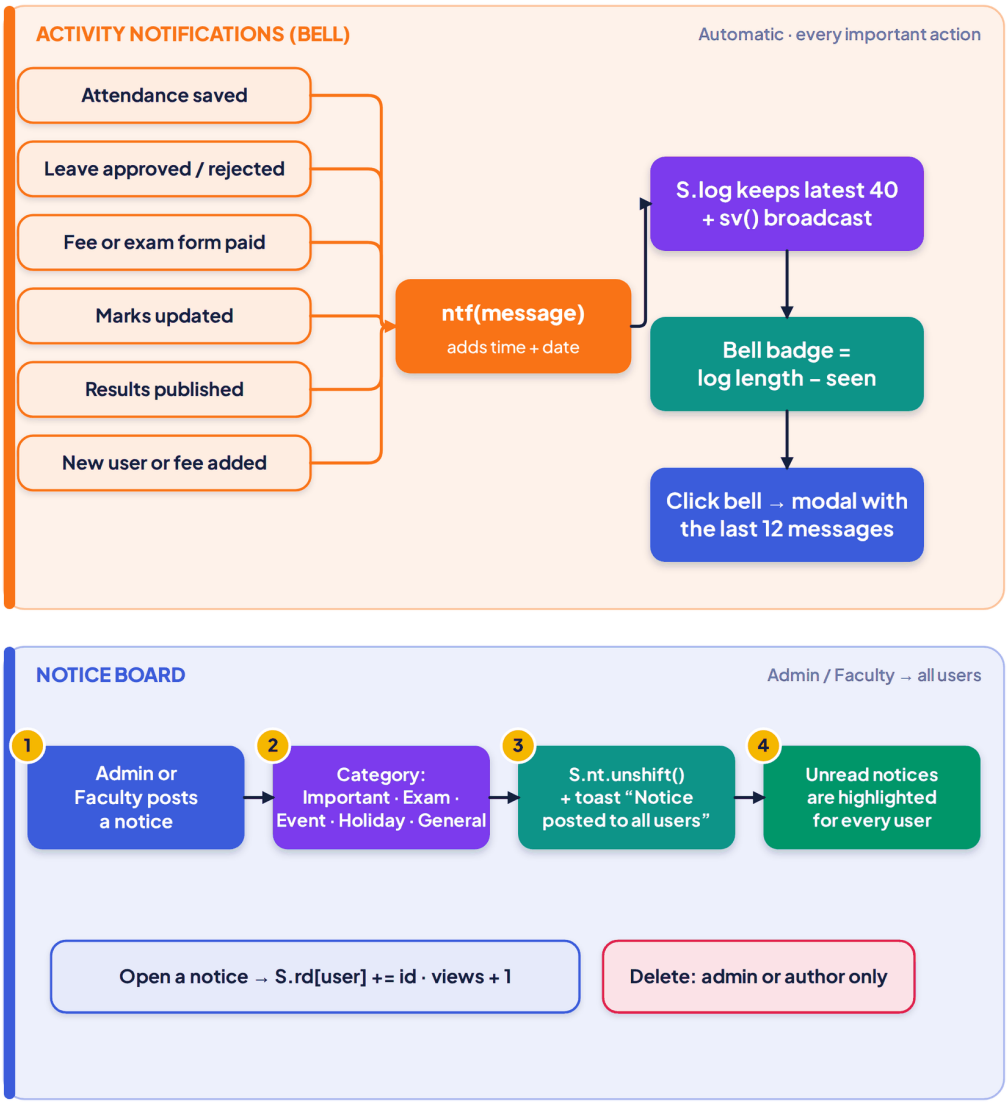


Figure 3.12: Notification bell and notice board

CHAPTER

4

SYSTEM IMPLEMENTATION

4.1 Project Structure

FOLDER STRUCTURE

```
RT-SMS-Student-Manegment-System/
├── index.html          ← login page
├── admin.html          ← admin portal shell
├── teacher.html        ← faculty portal shell
├── student.html        ← student portal shell
└── assets/
    ├── css/style.css   ← one light theme for all pages
    ├── images/rt-sms-logo.png
    └── js/
        ├── core.js     ← state, router, shell, helpers, sync
        ├── data.js     ← syllabus + demo seed data
        ├── auth.js     ← login logic
        ├── admin.js · teacher.js · student.js · shared.js
        ├── pdf.js      ← jsPDF documents
        └── logo.js     ← logo embedded for PDFs
```

4.2 Application Shell and Router

Each portal HTML file is almost empty. When the page loads, `boot(role)` verifies the session and calls `shell()`, which draws the logo, the role menu, the top bar with a live clock, the notification bell and the profile chip. A small router `go(page)` stores the current page key, highlights the menu item and calls the matching page function, so moving between modules never reloads the browser.

4.3 Important Functions

Function	File	What it does
ld() / sv()	core.js	Load the state from LocalStorage; save it and broadcast the change
sync()	core.js	Reload state in other tabs and soft re-render live pages
gr(total)	core.js	Return grade letter and grade points for a total mark
res(student, sem)	core.js	Marks, SGPA and credits for one semester
cgpa(student)	core.js	CGPA over all published semesters
rank(student, sem)	core.js	Class rank by SGPA
att(student, code)	core.js	Attendance percentage per subject
ntf(message)	core.js	Add an entry to the activity log (bell)
shell() / go() / render()	core.js	Draw layout, change page, redraw page
hdr() kv() pt() ftr()	pdf.js	Building blocks of every PDF document

4.4 Code Samples

Two short, representative parts of the project are shown below. The first turns total marks into a grade; the second saves data and informs the other tabs.

GRADE ENGINE

```
// grade and grade points from total marks (simplified)
const gr = t => t>=90 ? ["A+",10] : t>=80 ? ["A",9] : t>=70 ? ["B+",8]
             : t>=60 ? ["B",7]  : t>=50 ? ["C",6] : t>=40 ? ["D",5] : ["F",0];
```

LIVE SYNC

```
// save the state and tell every other tab
function sv(){
  localStorage.setItem(KEY, JSON.stringify(S));
  bc && bc.postMessage(1);              // BroadcastChannel
}
window.addEventListener("storage", sync); // fallback
```

4.5 PDF and QR Implementation

PDF files are created in the browser with jsPDF. The same helper functions draw the logo header, a student detail grid, a styled table with automatic page breaks and a footer, so every document has a consistent look. The UPI payment box uses qrcode-generator to draw a QR code for the exact amount.

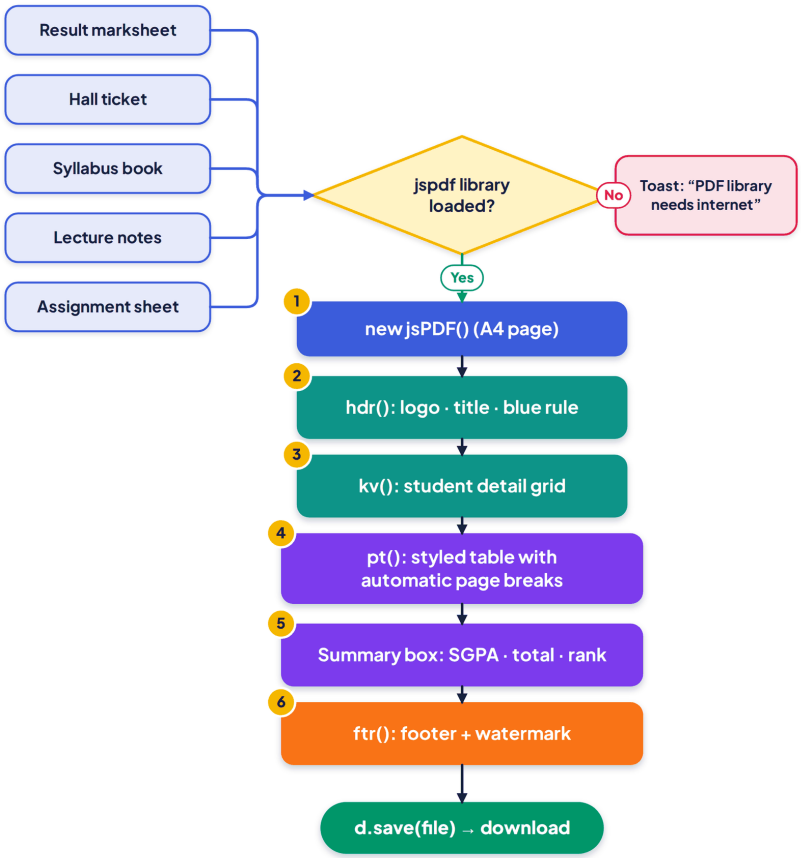


Figure 4.1: PDF generation pipeline

4.6 Development and Deployment

The project was built step by step: requirements, interface design, data design, core engine, the three portals, PDF and payments, and finally testing. Each step was checked in the browser before the next began. The finished files are pushed to GitHub, and GitHub Pages serves them directly as a static website.

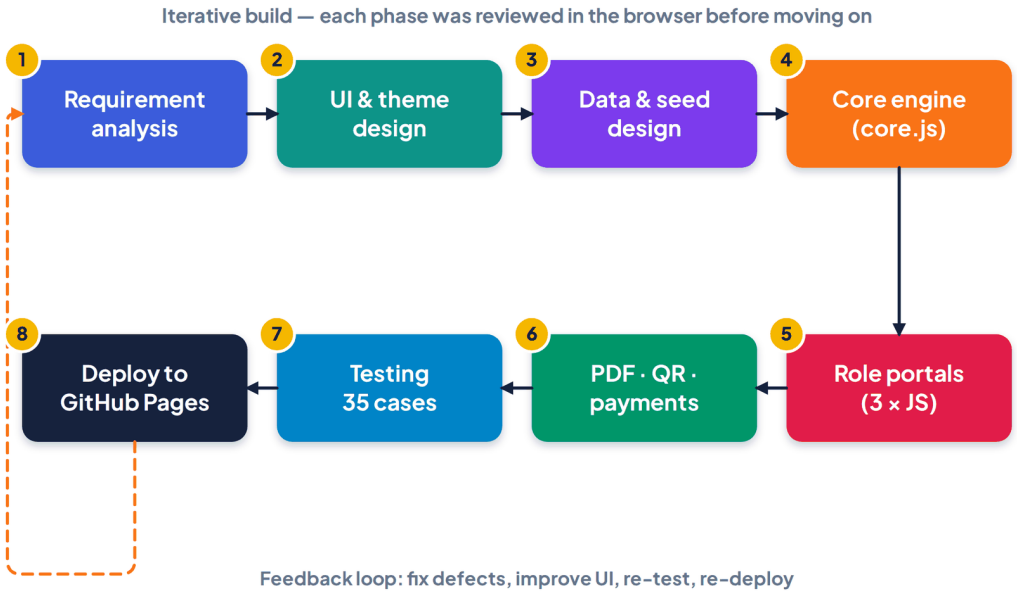


Figure 4.2: Development lifecycle

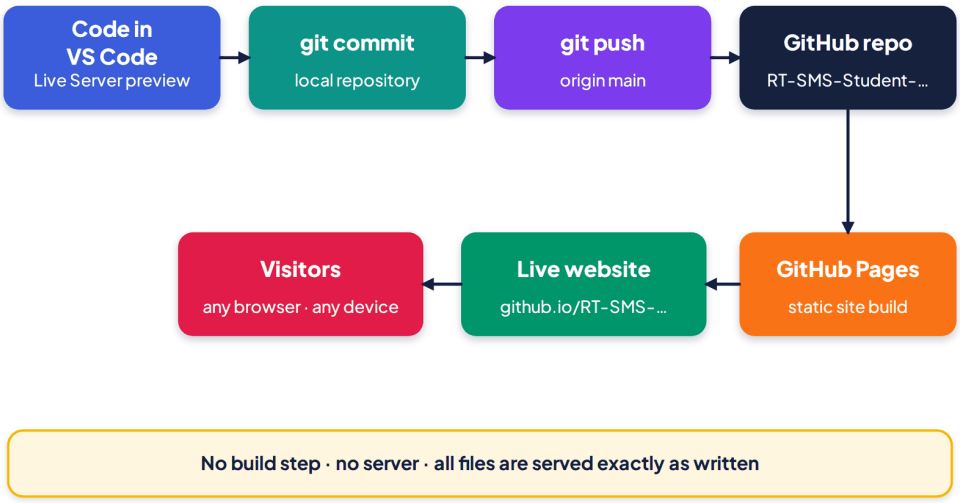


Figure 4.3: Deployment workflow with GitHub Pages

4.7 Security and Validation Notes

- Every portal checks the session, the role and the account status before showing any page.
- Forms validate input: leave needs a reason and valid dates, card payment checks card details, marks are limited to 0–30 and 0–70.
- Admin can disable an account; a disabled user cannot log in.
- Because the project is front-end only, passwords and data live in the browser. This is suitable for a demonstration and learning project; a real deployment would add a server, hashed passwords and secure sessions (see Chapter 8).

CHAPTER

5

SOFTWARE TESTING

The project was tested with **35 automated test cases** that drive a real Chromium browser with Playwright. The tests start from a fresh data set, perform the same clicks a user would perform, and then check both the screen and the stored data. Every test of every group passed in the final run, and no JavaScript error was reported during the run.

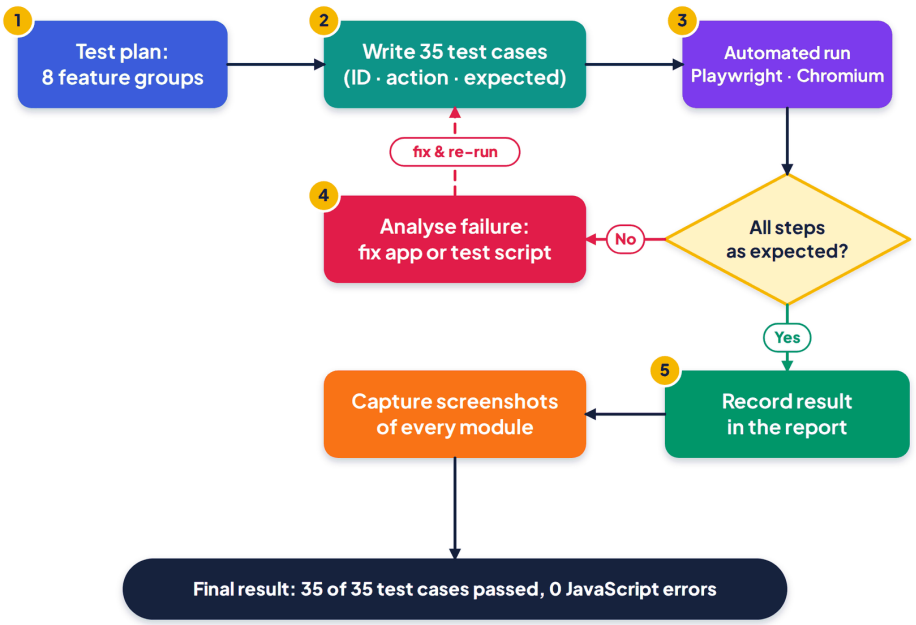


Figure 5.1: Testing process

5.1 Authentication and Access Control

ID	Test Case	Expected Result	Observed	Result
AU-01	Student login with demo account	Redirects to student portal	Opened student.html	PASS
AU-02	Faculty login with demo account	Redirects to faculty portal	Opened teacher.html	PASS
AU-03	Admin login with demo account	Redirects to admin portal	Opened admin.html	PASS
AU-04	Login with wrong password	Error toast, stays on login	Invalid credentials for this role	PASS
AU-05	Student credentials on Admin tab	Login rejected (role mismatch)	Student e-mail rejected on Admin tab	PASS
AU-06	Open admin.html without a session	Redirect to login	Redirected to login page	PASS

ID	Test Case	Expected Result	Observed	Result
AU-07	Student session opens admin.html	Redirect to login (role check)	Student session blocked from Admin page	PASS
AU-08	Logout	Session cleared, back to login	Session cleared, login shown	PASS

5.2 Attendance and Leave

ID	Test Case	Expected Result	Observed	Result
AT-01	Faculty marks attendance (Present/Late/Absent) and saves	Saved to storage with success toast	Toast: “Attendance saved & synced live”	PASS
AT-02	Admin live feed updates without reload (2 tabs)	New entry appears on Admin tab within 1 s	Feed top row updated by BroadcastChannel	PASS
AT-03	Student dashboard reflects today’s status	Subject chip shows the new status	Student chip shows Absent	PASS
AT-04	Change minimum-attendance threshold in Settings	Low-attendance alert list recalculates	Alert rows 1 → 11 when minimum set to 100%	PASS
LV-01	Student applies for leave	Request saved with status Pending	New request saved as Pending	PASS
LV-02	Faculty approves the request	Student sees Approved with approver name	Student history shows Approved by Neha Mam	PASS

5.3 Exam Form, Fees and Results

ID	Test Case	Expected Result	Observed	Result
EX-01	Exam fee calculation on form	Total = ₹1,950 + ₹500 per subject	5 subjects → ₹4450 (1950 + 500 × 5)	PASS
EX-02	Submit exam form and pay via Net Banking	Status Paid; hall ticket unlocked	Payment successful – Hall Ticket button unlocked	PASS
EX-03	Download hall ticket PDF	PDF file generated by jsPDF	Downloaded HallTicket_23SS02IT001.pdf	PASS
FE-01	Admin adds a fee for one student	Student ledger shows the new Due fee	Admin fee visible to student as Due (₹750)	PASS
FE-02	Card payment validation	Bad card rejected, valid card accepted	Invalid card rejected; valid card marks fee Paid	PASS
RS-01	Faculty enters Internal /30 and External /70	Total and grade computed (28+65 → 93, A+)	28 + 65 = 93 → Grade A+ shown	PASS
RS-02	Admin publishes semester 7 results	Student sees marksheet, SGPA, CGPA and rank	Semester 7 results visible with SGPA, CGPA, rank	PASS
RS-03	Download result PDF	Statement-of-marks PDF generated	Downloaded Result_23SS02IT001_Sem7.pdf	PASS
RS-04	Grade-point boundaries (39, 40, 50 … 90, 100)	F/D/C/B/B+/A/A+ with 0–10 points	13 boundary marks mapped correctly (F<40 … A+≥90)	PASS

5.4 Notices, Grievance, Admin and Responsive UI

ID	Test Case	Expected Result	Observed	Result
NT-01	Admin posts a notice	Notice appears for students; bell counter increases	Notice visible to student; bell counter = 11	PASS
GR-01	Student raises grievance; admin resolves it	Status and response visible to student	Student sees Resolved + admin response	PASS

ID	Test Case	Expected Result	Observed	Result
FB-01	Student rates faculty (4 criteria)	Feedback saved with confirmation	Toast: “Thanks for your feedback”	PASS
AD-01	Add a new student	Record added with auto ID	Rows 12 → 13; new ID 23SS02IT013 auto-generated	PASS
AD-02	Disable a user account	Disabled user cannot log in	Toast: “Account disabled – contact Admin”	PASS
AD-03	Export full backup (JSON)	Valid JSON containing all collections	backup.json valid (22 users)	PASS
AD-04	Export students as CSV	CSV downloaded	Downloaded students.csv	PASS
UI-01	Desktop (1366 px)	Sidebar always visible	Sidebar visible, menu button hidden at 1366 px	PASS
UI-02	Mobile (390 px) menu	Sidebar collapses; ☰ toggles it	Sidebar hidden; ☰ opens it	PASS
UI-03	Mobile page overflow (4 pages)	No sideways scrolling	No horizontal page scroll on 4 student pages at 390 px	PASS
UI-04	Tablet (768 px) admin dashboard	Layout adapts without overflow	Admin dashboard fits at 768 px (tablet)	PASS
UI-05	Console / runtime errors during whole run	Zero JavaScript errors	0 JavaScript errors across all tested pages	PASS

5.5 Test Case Summary

Test Category	Total	Passed	Failed
Authentication	8	8	0
Attendance	4	4	0
Leave	2	2	0
Exam form & fees	5	5	0
Results	4	4	0
Notices	1	1	0
Grievance & feedback	2	2	0
Admin management	4	4	0
Responsive UI	5	5	0
Total	**35**	**35**	**0**

AUTOMATED CHECK RESULT

All 35 test cases passed and the browser console showed **0 JavaScript errors**. During preparation, four checks first failed because of mistakes in the test script itself (a hidden radio button and a page-hash navigation); the application behaved correctly and the script was corrected and re-run.

CHAPTER

6

SCREENSHOTS AND WALKTHROUGH

The screenshots below were captured from the running project (the same code that is published on GitHub Pages). The address bar of every frame shows the matching page of the live website. The records shown are demo data.

6.1 Login Page

The login page offers three role tabs and a quick-fill list of demo accounts, so any visitor can try every portal in seconds.

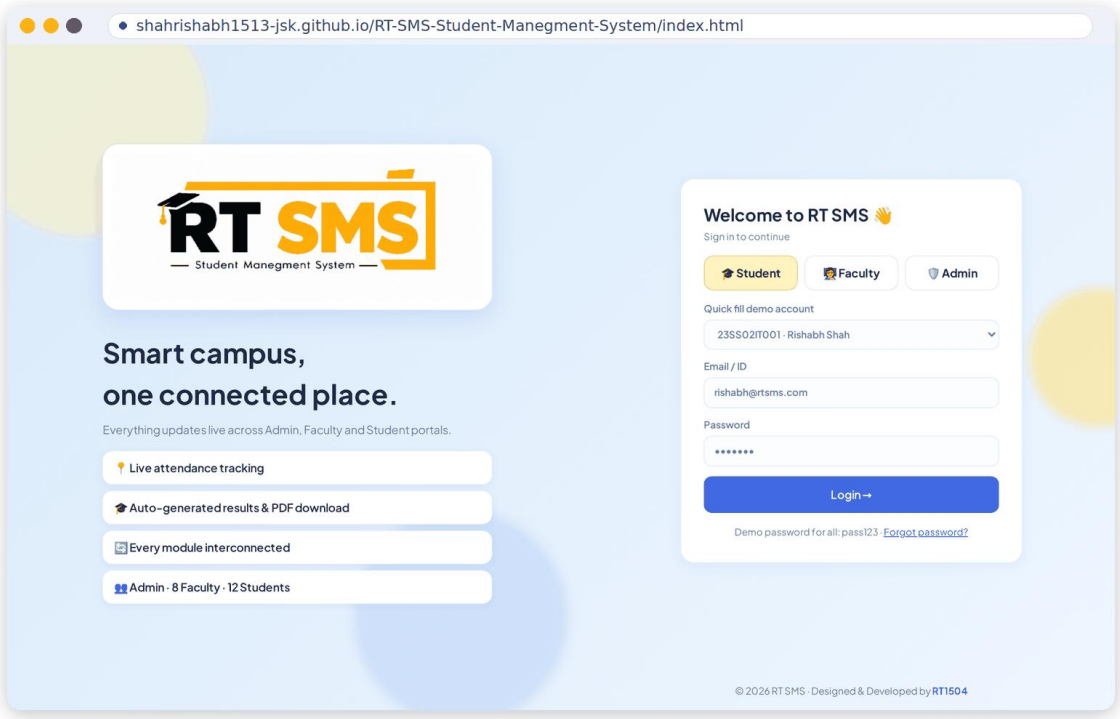


Figure 6.1: Login page with role tabs and demo accounts

6.2 Admin Portal

The Admin Control Center shows student and faculty counts, today's attendance, pending approvals, live attendance bars and a live feed.

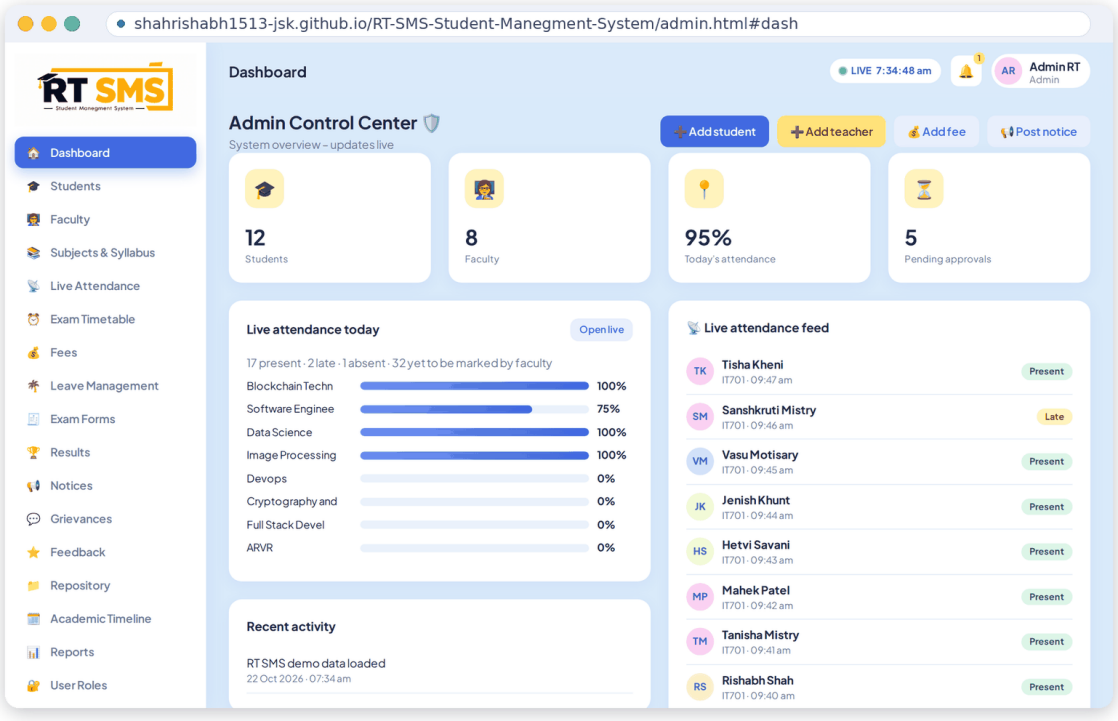


Figure 6.2: Admin dashboard – live attendance and feed

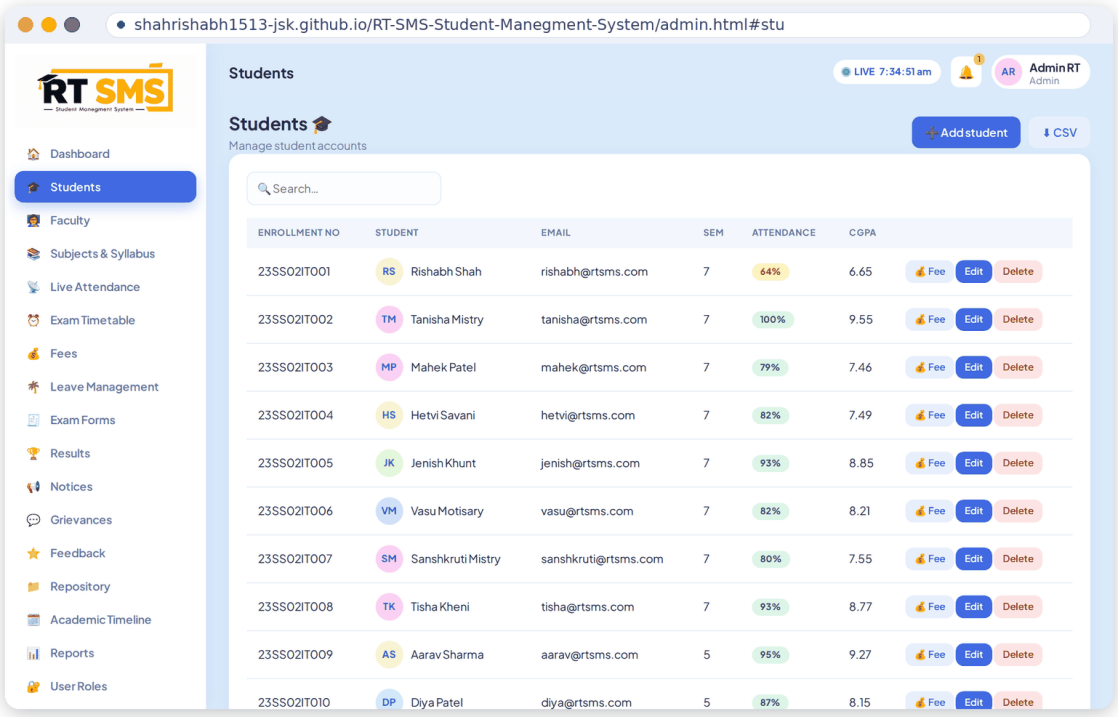


Figure 6.3: Students management

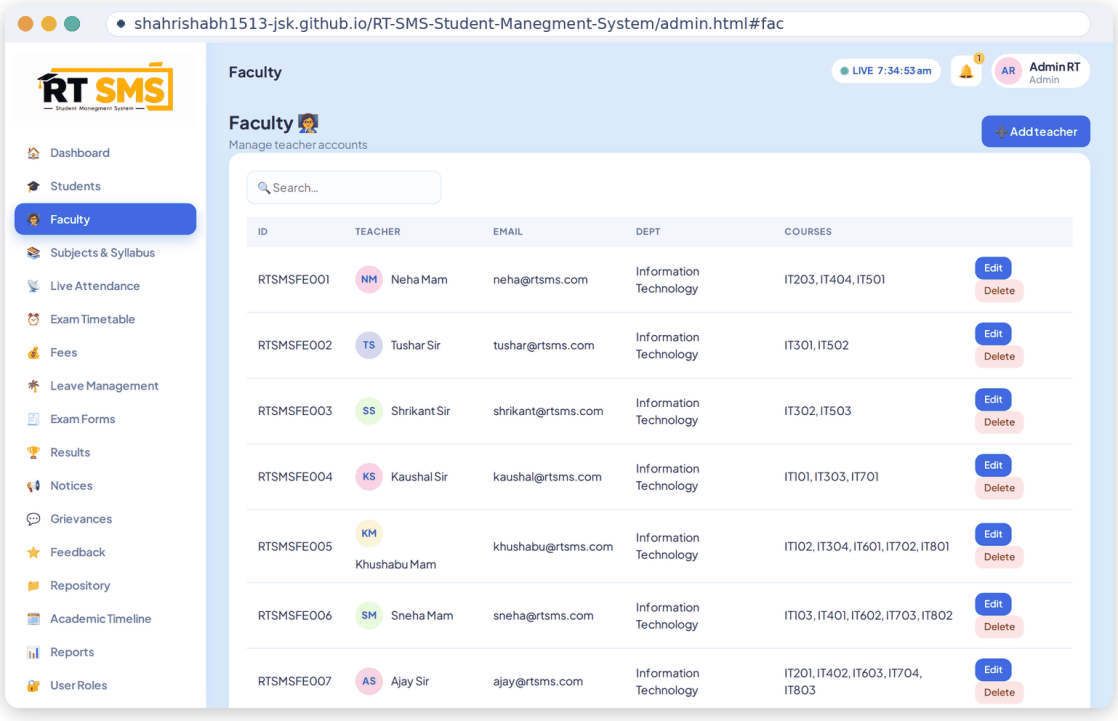


Figure 6.4: Faculty management

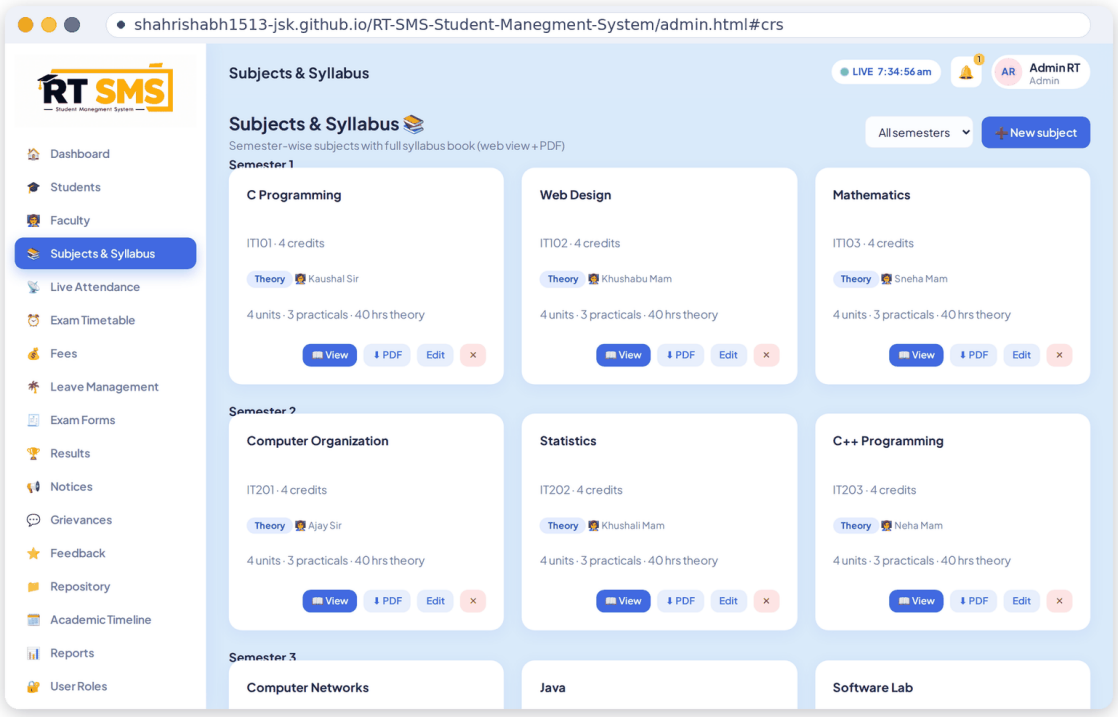


Figure 6.5: Subjects and syllabus

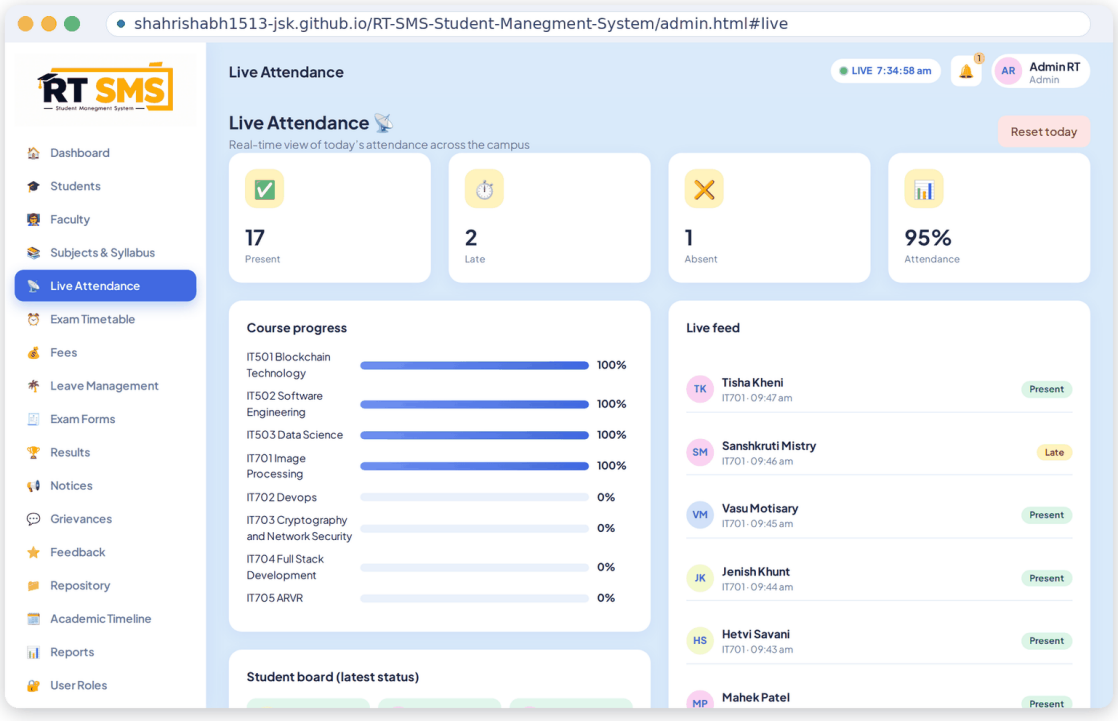


Figure 6.6: Live attendance (admin)

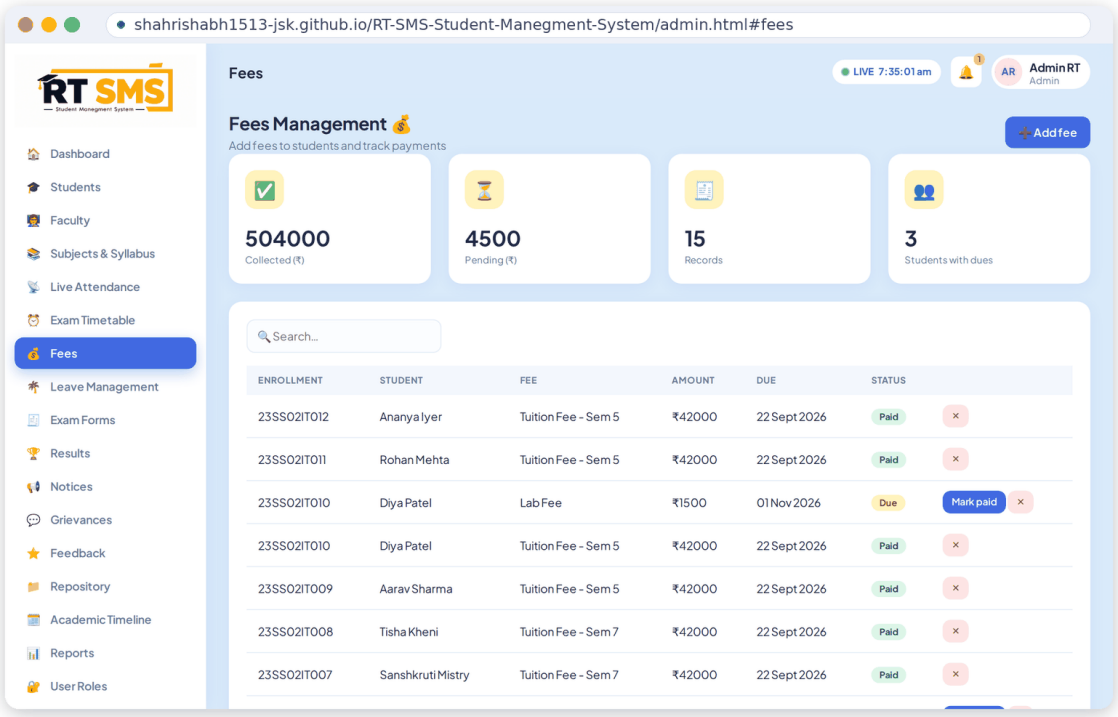


Figure 6.7: Fee management

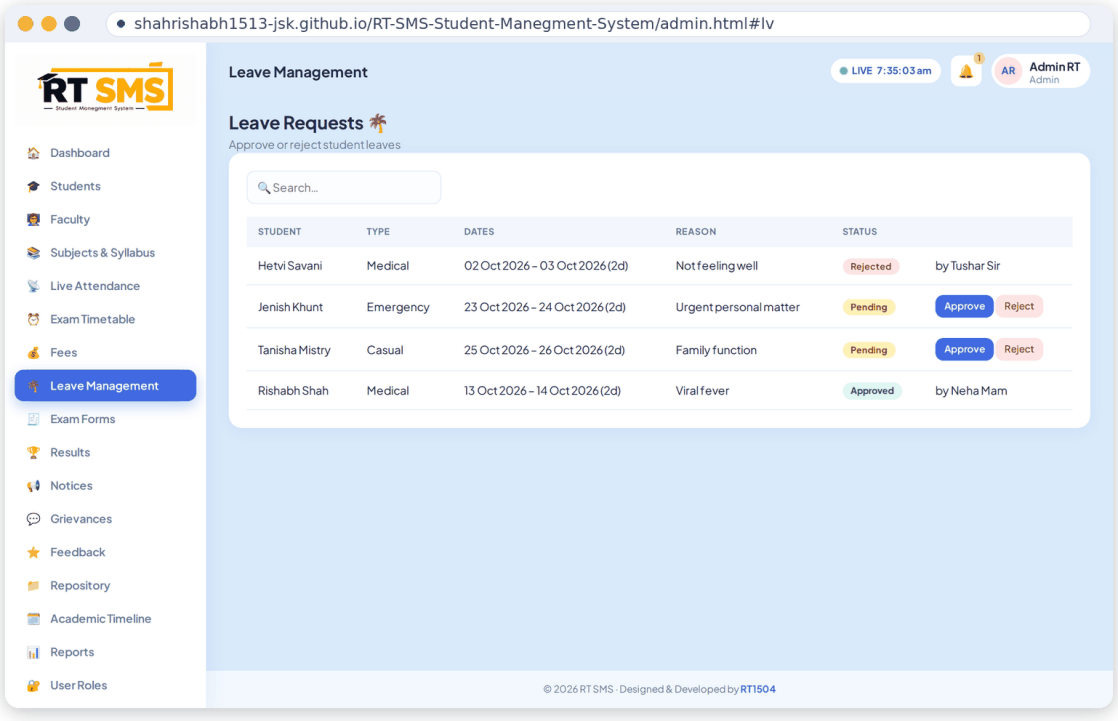


Figure 6.8: Leave management

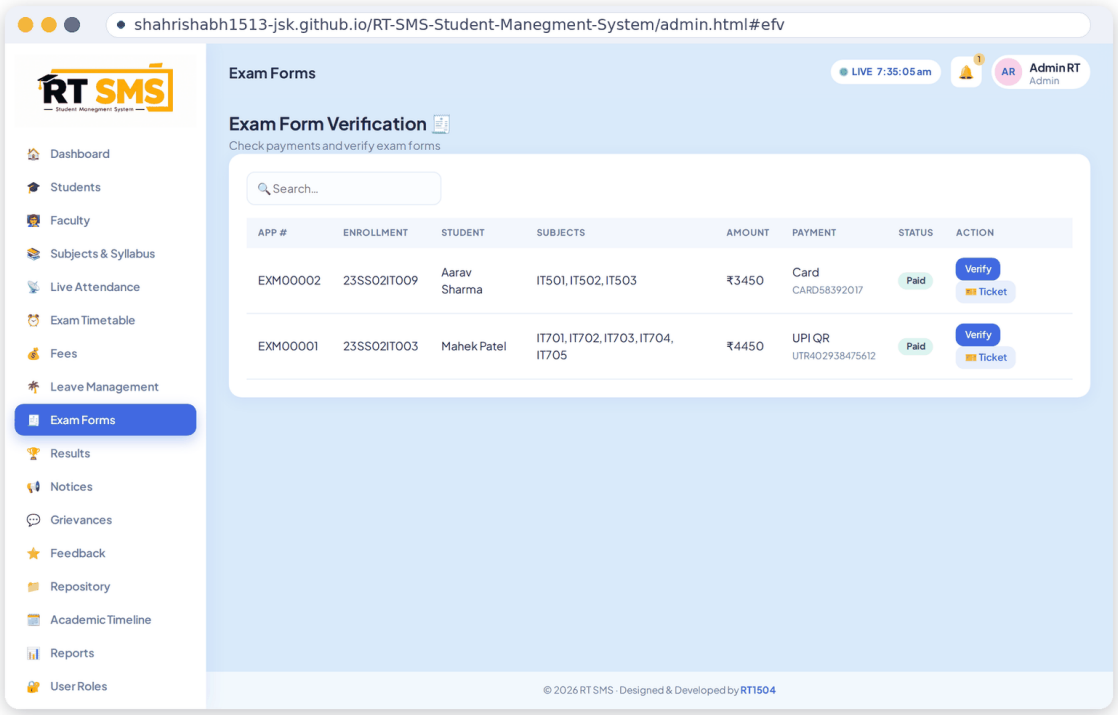


Figure 6.9: Exam forms and payment status

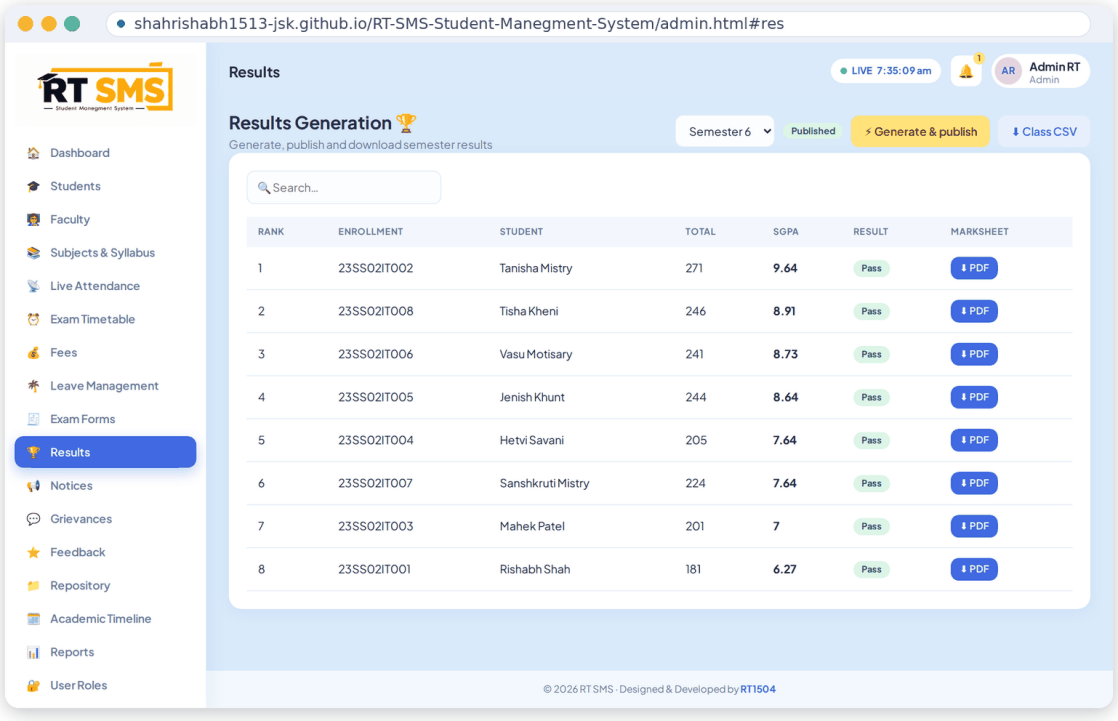


Figure 6.10: Results generation and publishing

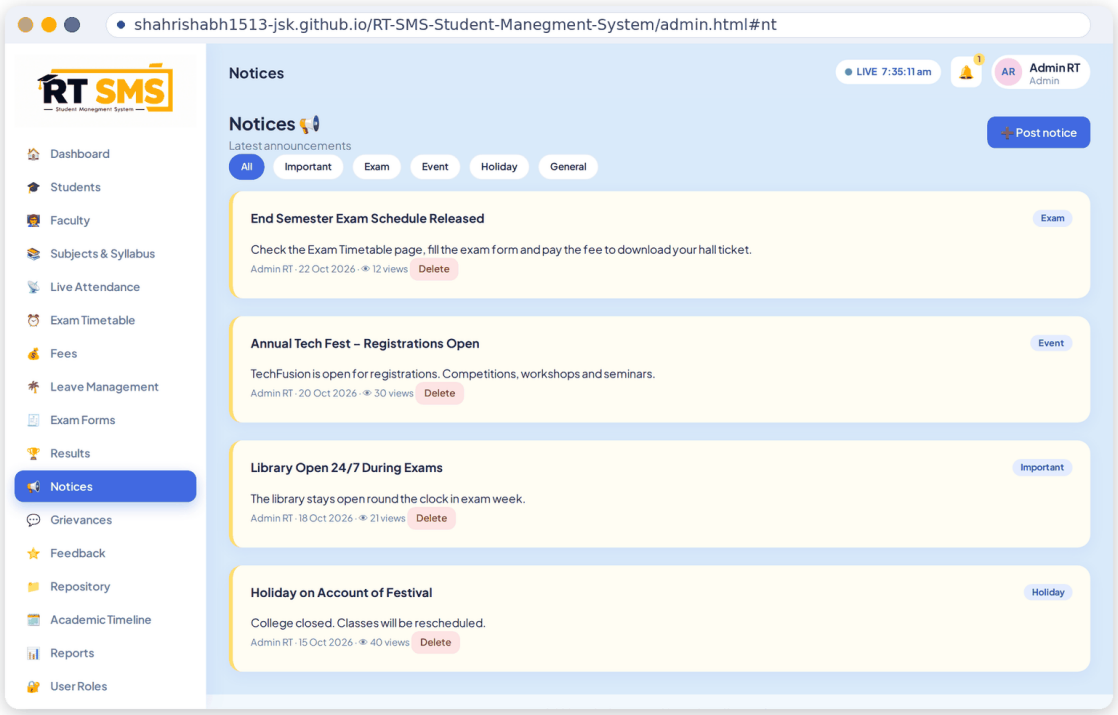


Figure 6.11: Notices

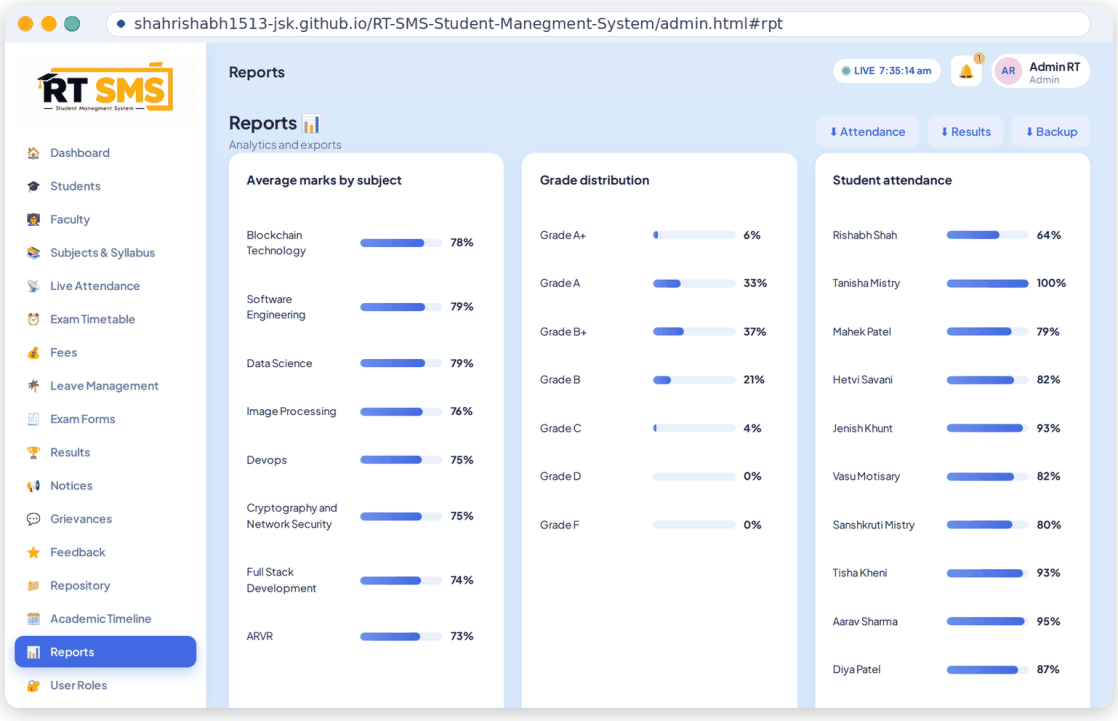


Figure 6.12: Reports and grade distribution

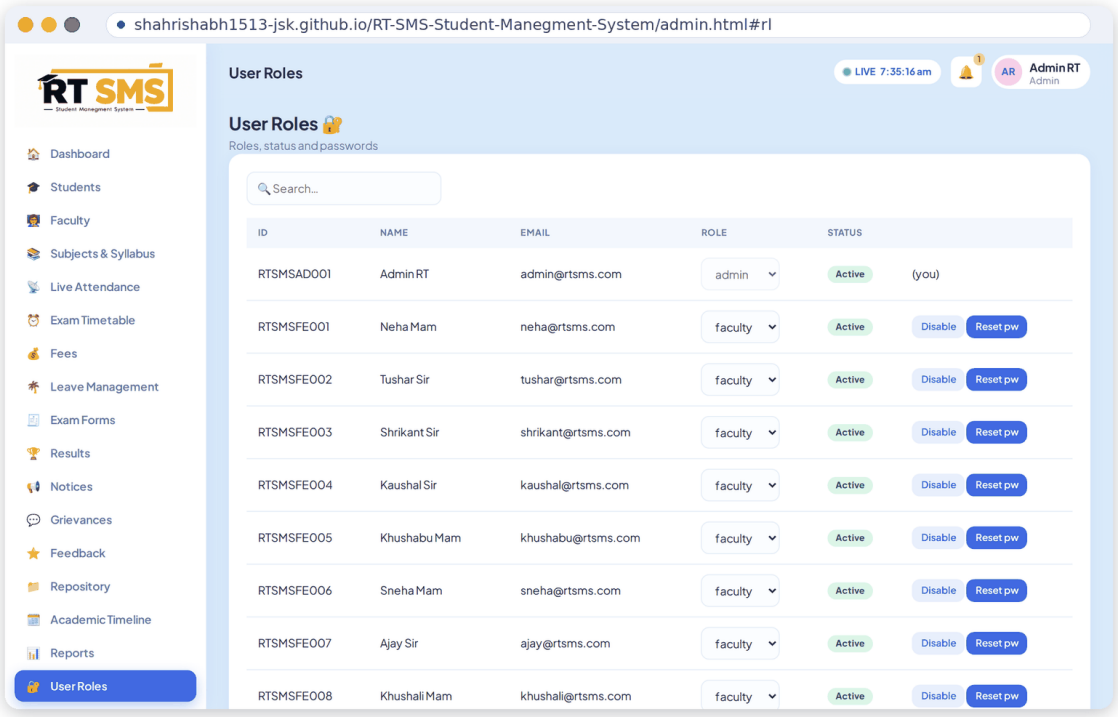


Figure 6.13: User roles

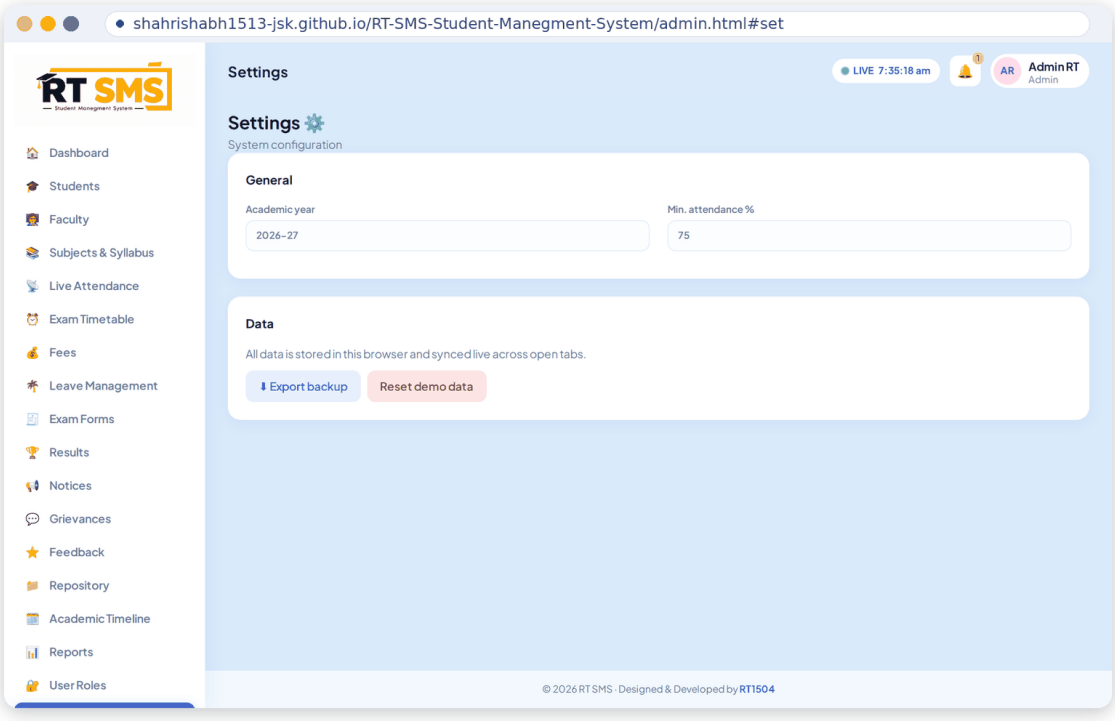


Figure 6.14: Settings, backup and reset

6.3 Faculty Portal

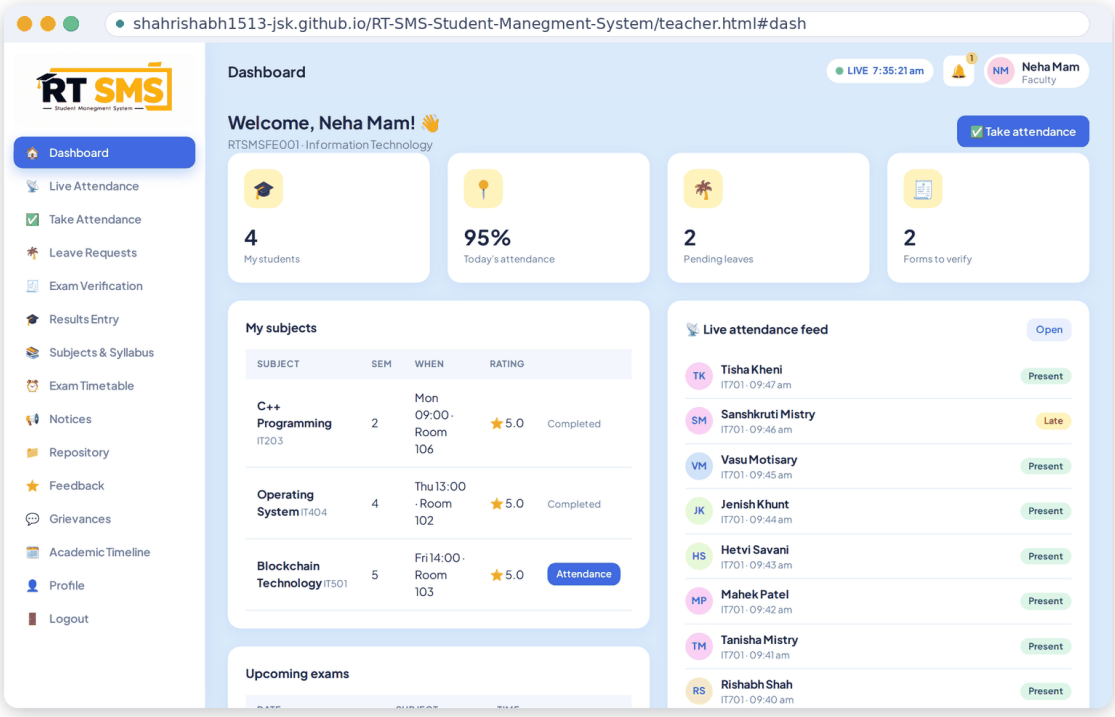


Figure 6.15: Faculty dashboard

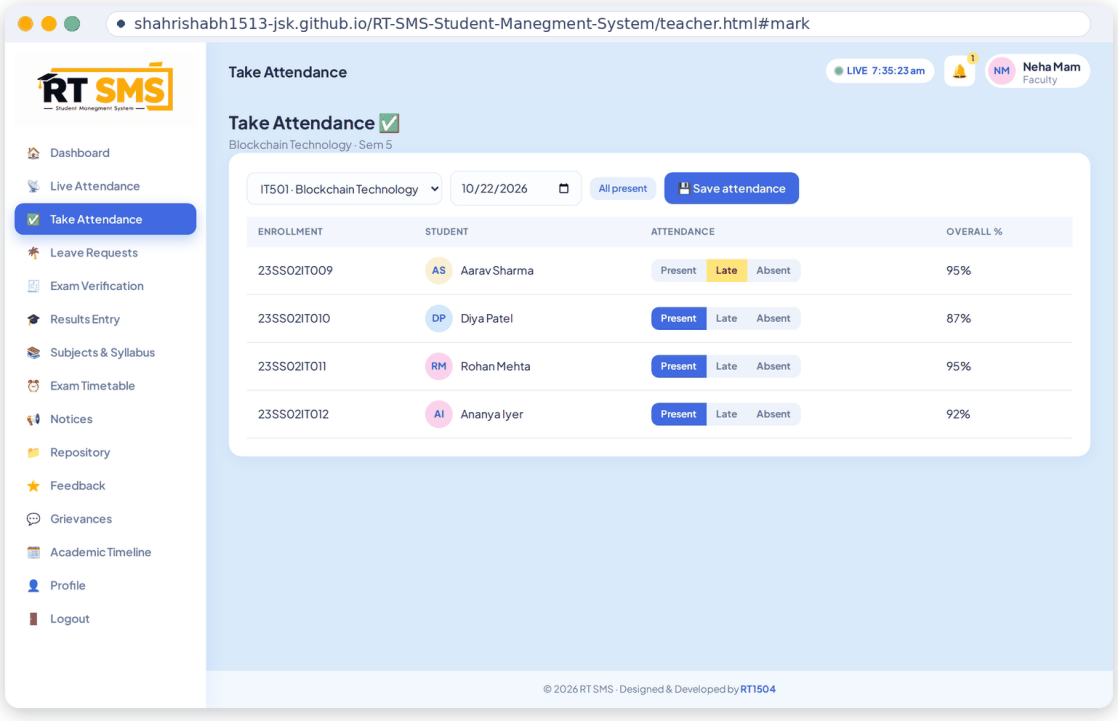


Figure 6.16: Take attendance

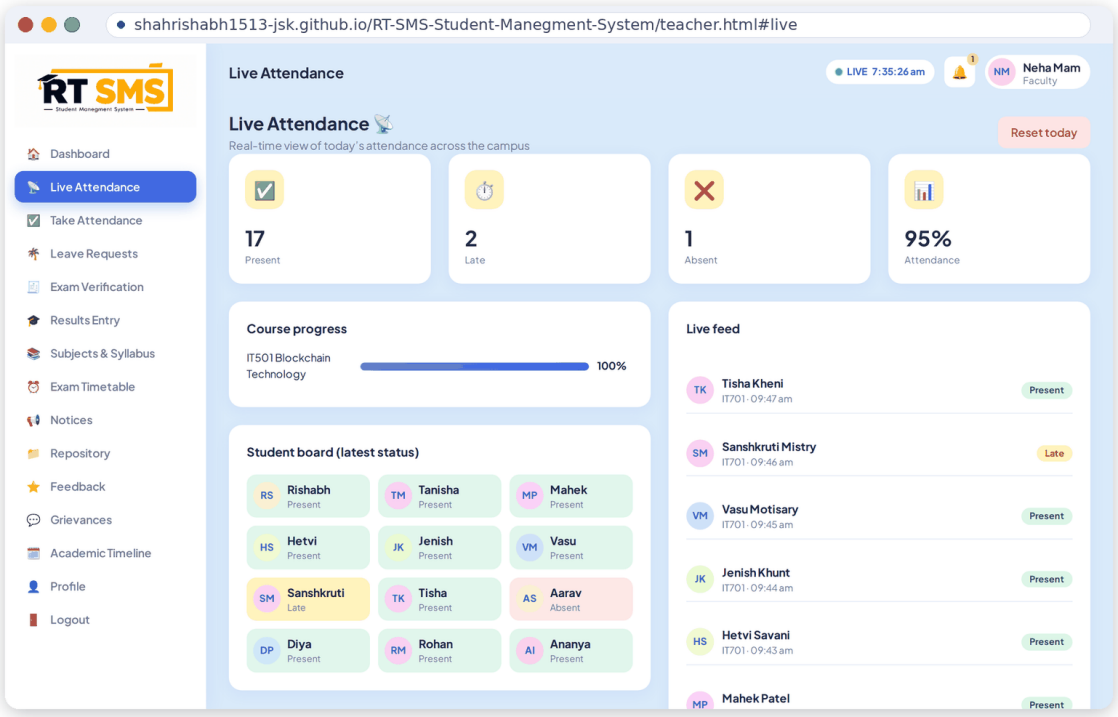


Figure 6.17: Live attendance (faculty)

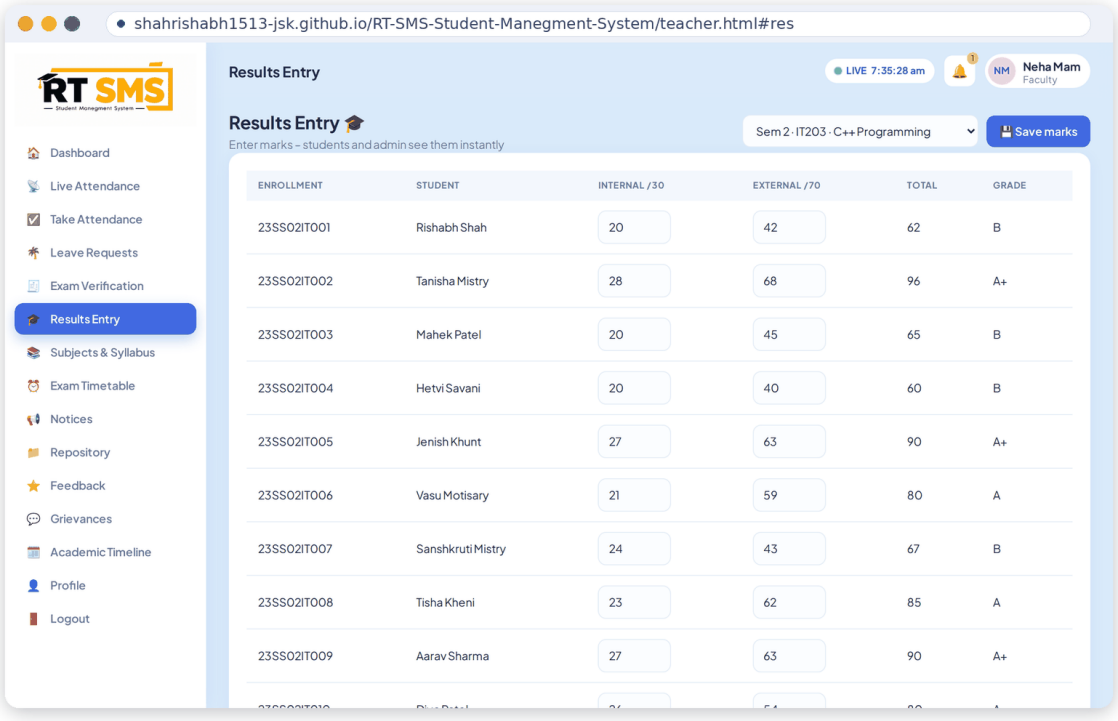


Figure 6.18: Results entry – internal and external marks

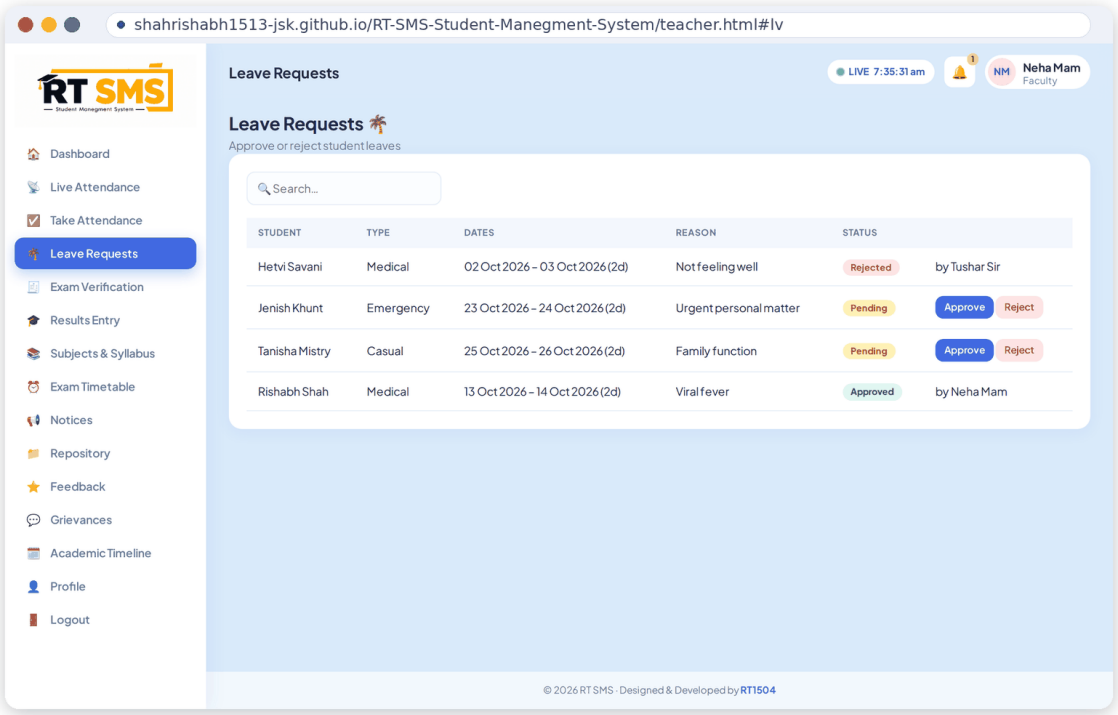


Figure 6.19: Leave requests

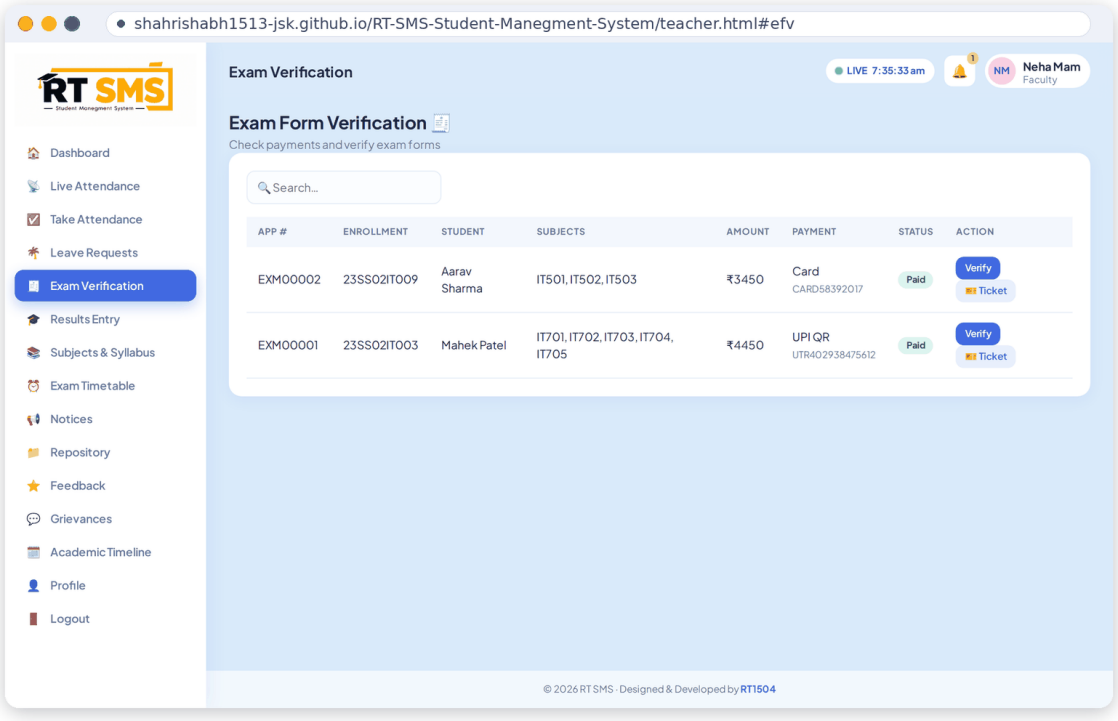


Figure 6.20: Exam form verification

6.4 Student Portal

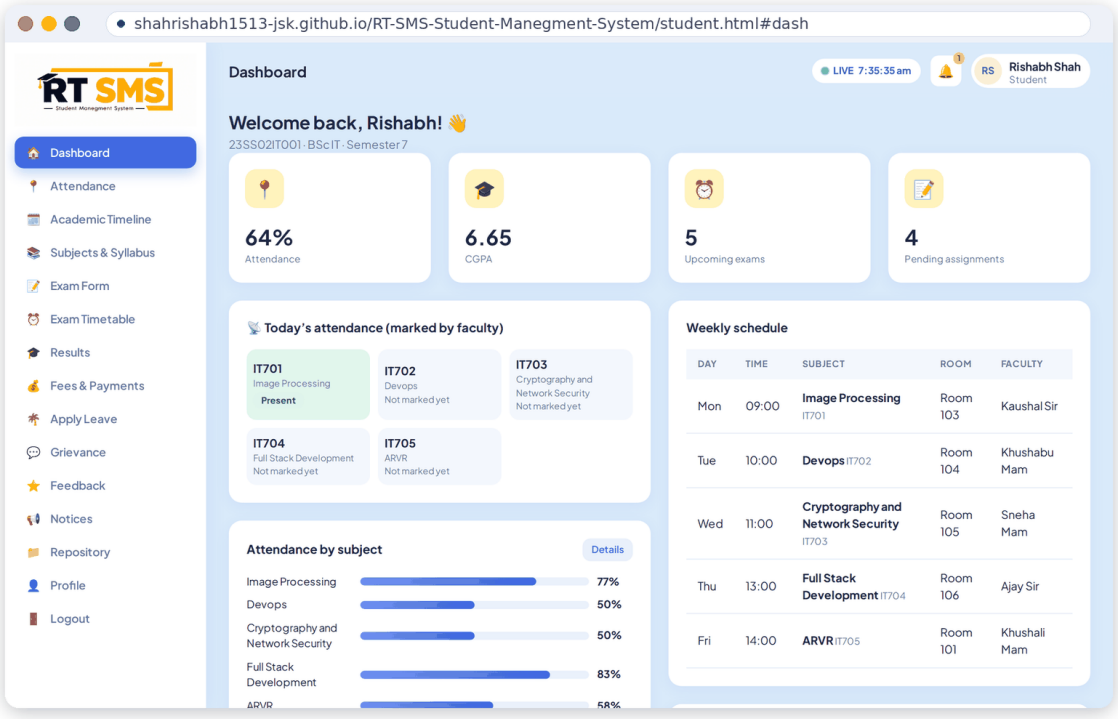


Figure 6.21: Student dashboard

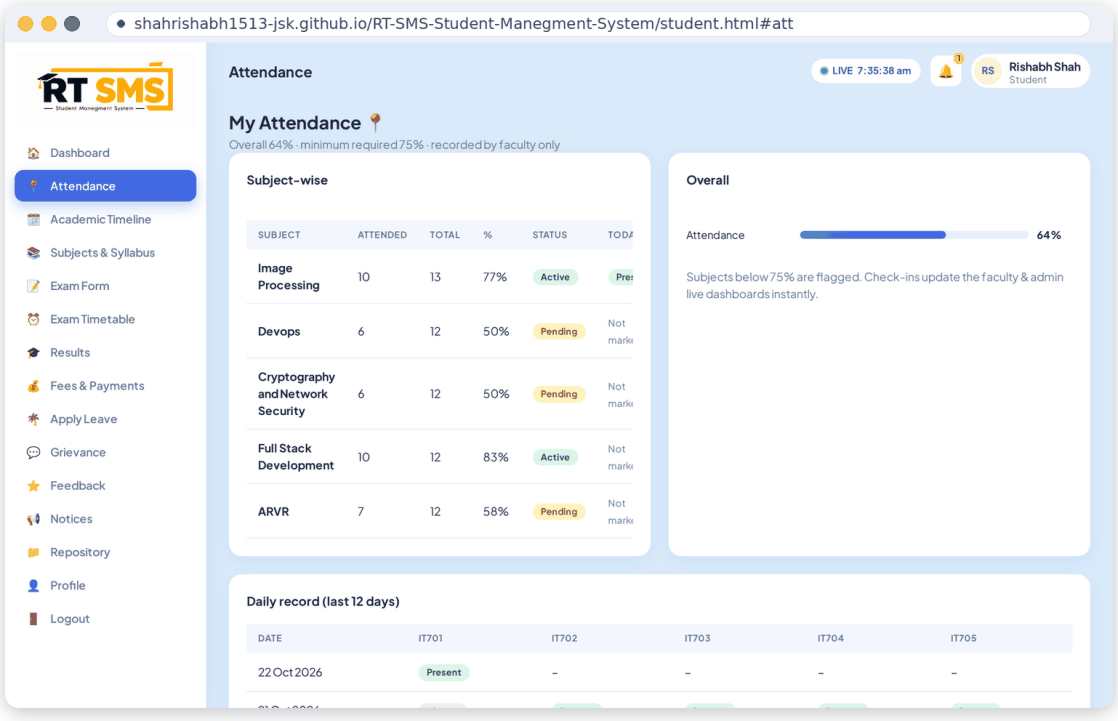


Figure 6.22: Attendance by subject

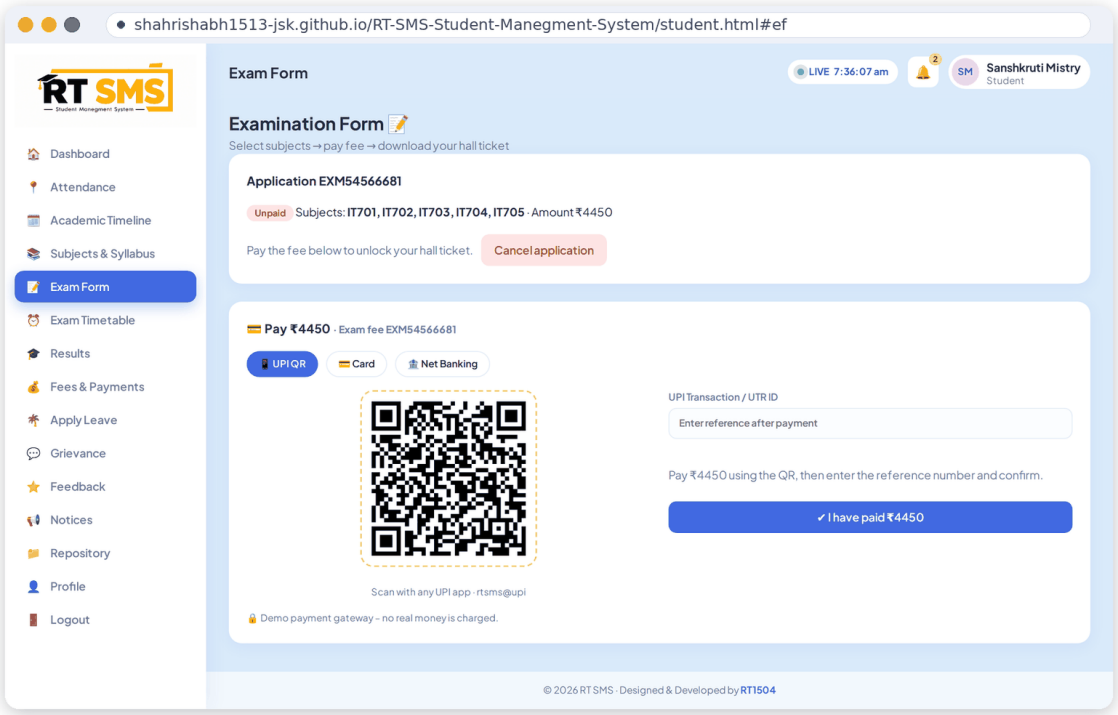


Figure 6.23: Exam form and payment

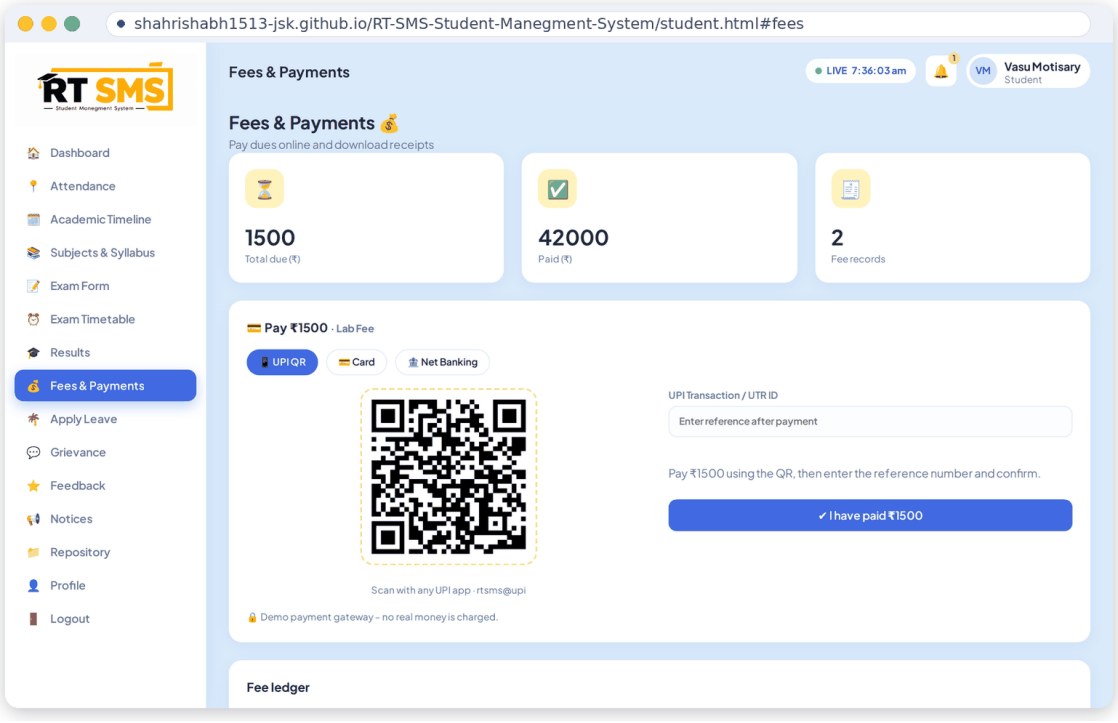


Figure 6.24: Fee payment with UPI QR code

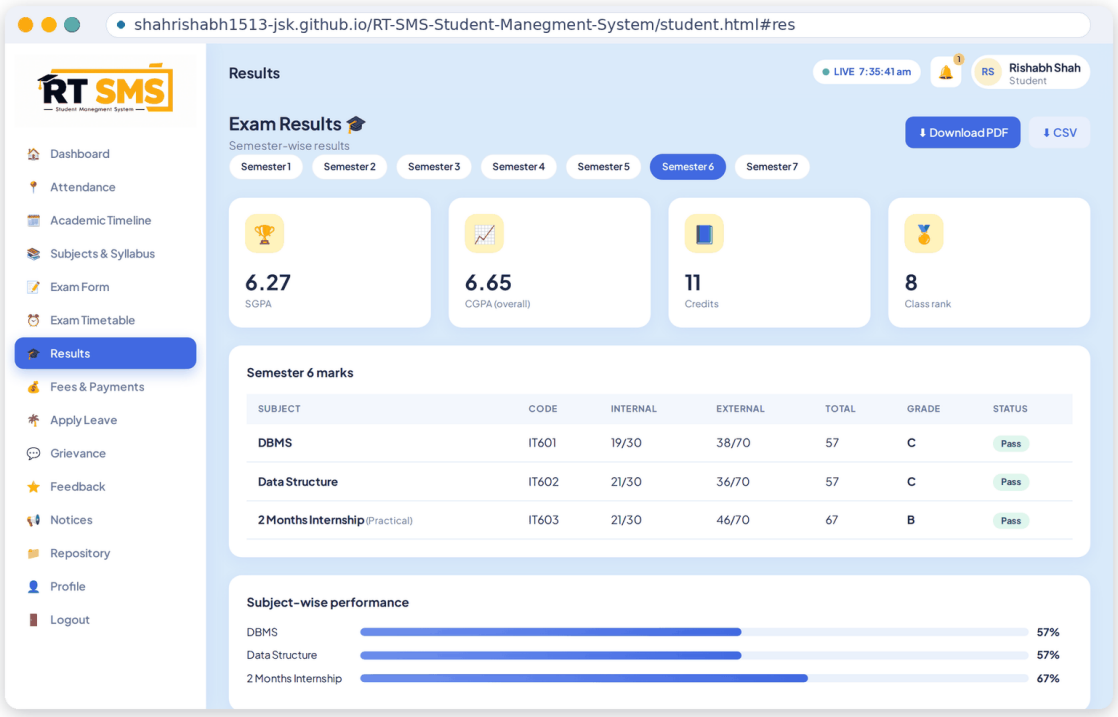


Figure 6.25: Results – SGPA, CGPA and rank

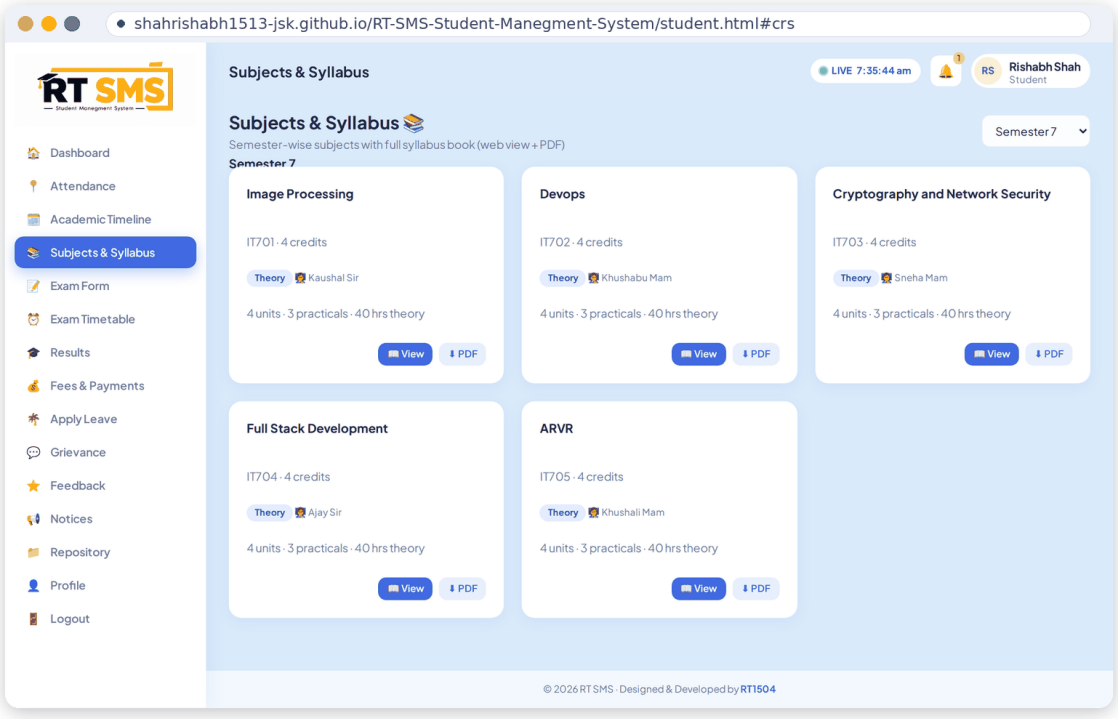


Figure 6.26: Subjects and syllabus

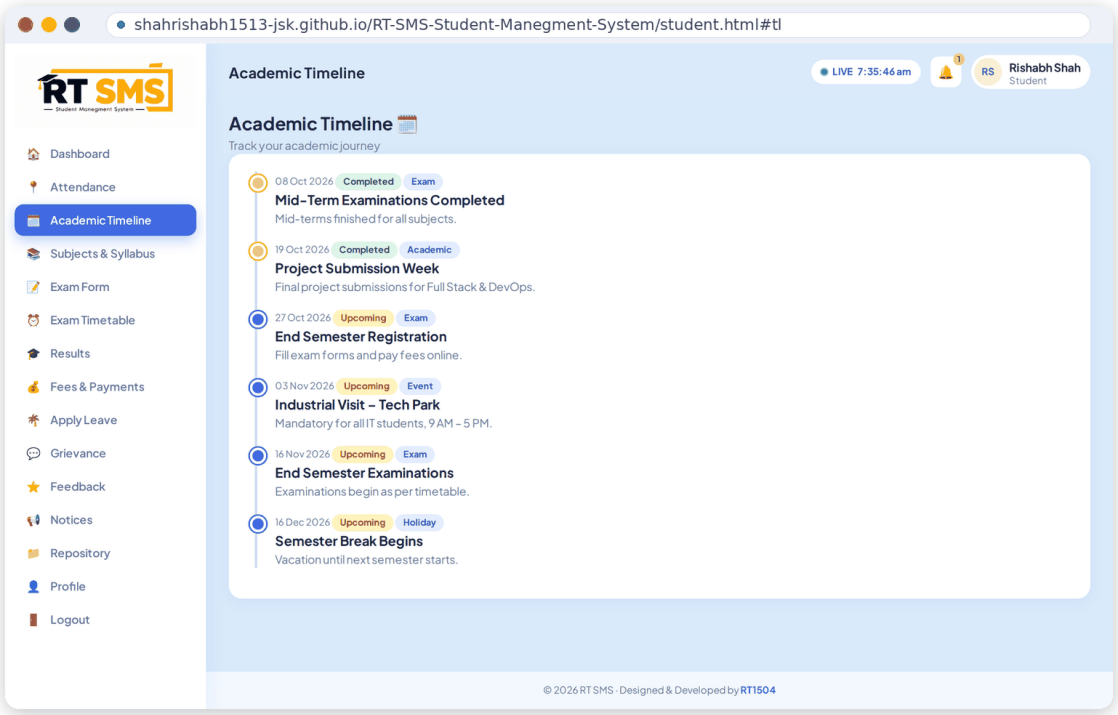


Figure 6.27: Academic timeline

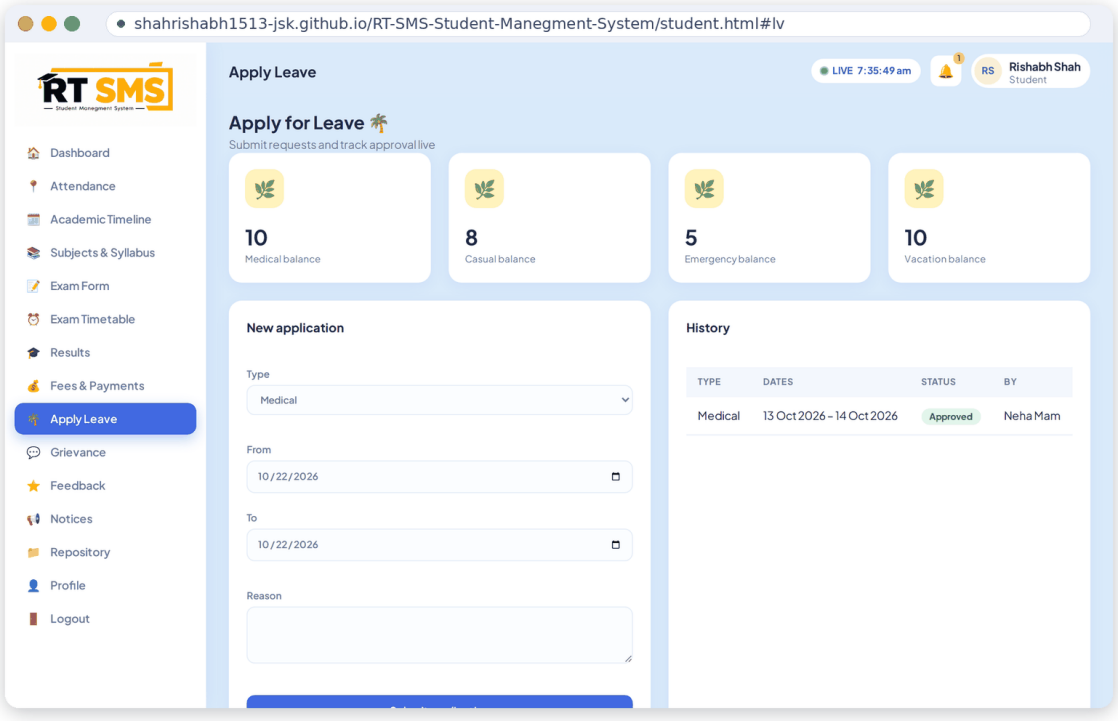


Figure 6.28: Apply for leave

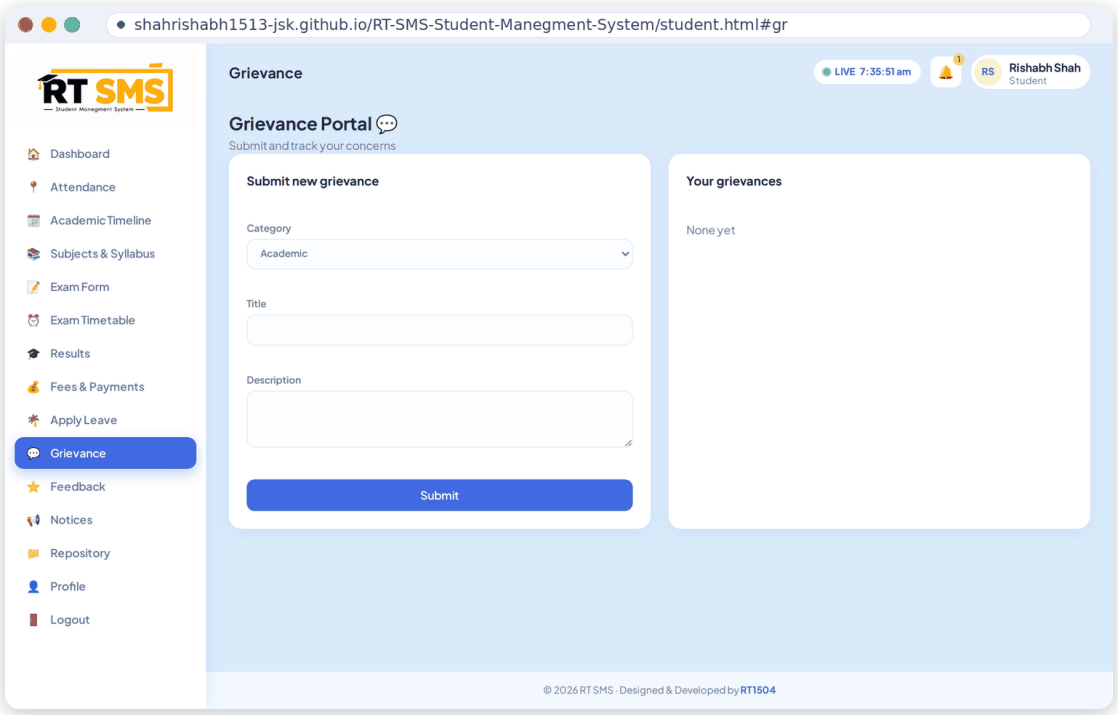


Figure 6.29: Grievance portal

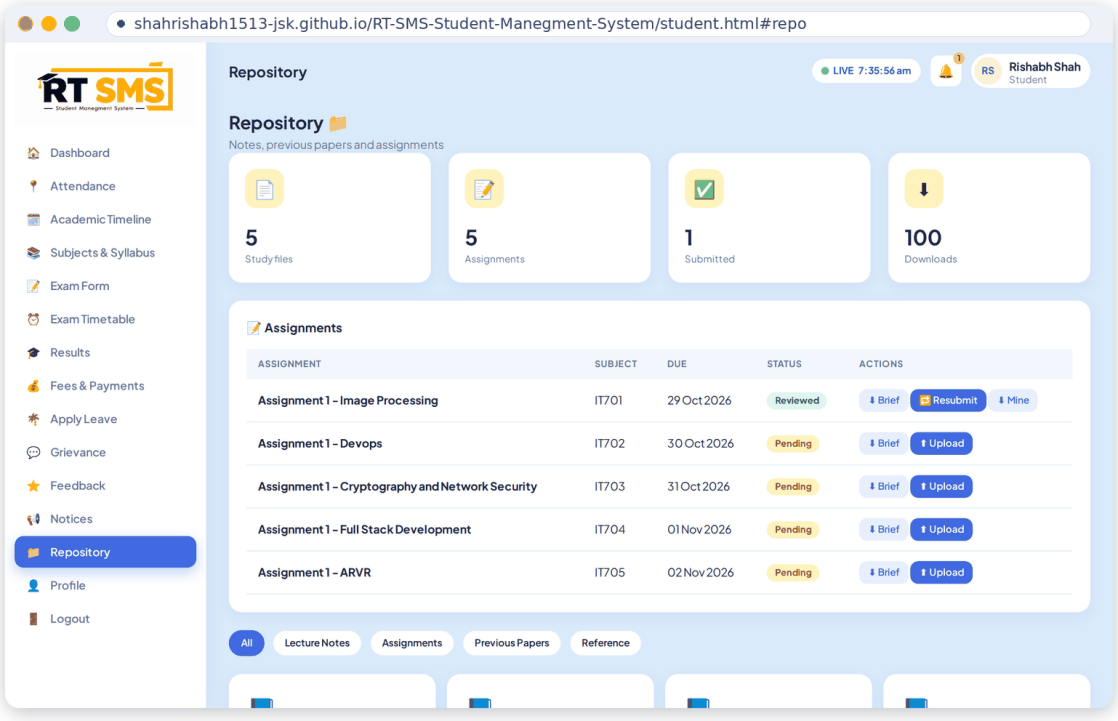


Figure 6.30: Repository – notes and assignments

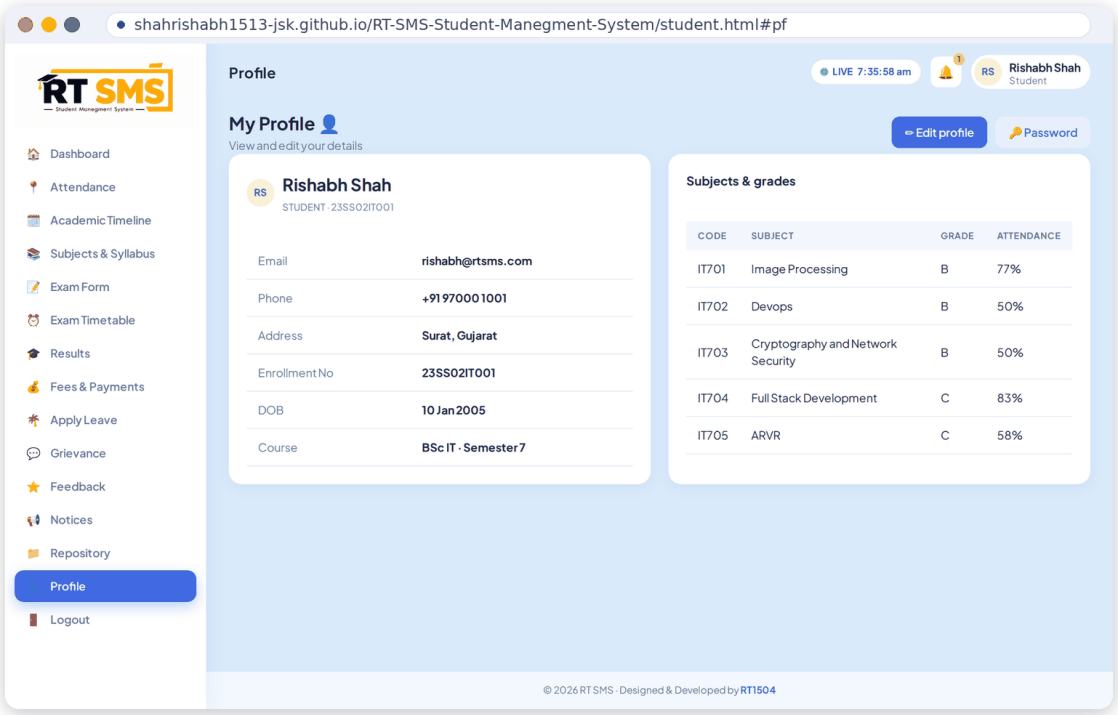


Figure 6.31: Student profile

6.5 Generated PDF Documents

Hall ticket and result marksheet are generated in the browser with jsPDF and downloaded as PDF files.



HALL TICKET

End Semester Examination - 2026-27

RT SMS

Student Name: Rishabh Shah
Enrollment No: 23SS02IT001
Semester: 7
Application No: EXM54389135
Exam Title: Theory Examination
Fee Status: PAID (NB654390763)

Subject Name	Code	Type	Date / Timing	Class Room	Faculty Signature
Image Processing	IT701	Theory	01 Nov 2026 10:00 - 13:00	Room 103	
Devops	IT702	Theory	02 Nov 2026 10:00 - 13:00	Room 104	
Cryptography and Network Security	IT703	Theory	03 Nov 2026 10:00 - 13:00	Room 105	
Full Stack Development	IT704	Theory	04 Nov 2026 10:00 - 13:00	Room 106	
ARVR	IT705	Theory	05 Nov 2026 10:00 - 13:00	Room 101	

Student Signature

Controller of Examinations

Instructions to Candidates

1. Carry this hall ticket and your college ID card to every examination.
2. Report to the examination room 15 minutes before the scheduled time.
3. Mobile phones, smart watches and electronic gadgets are strictly prohibited.
4. No candidate is allowed to leave the hall during the first 30 minutes.
5. Use of unfair means will lead to cancellation of the examination.
6. Get the invigilator / faculty signature on this hall ticket during the exam.

(c) 2026 RT SMS - Designed & Developed by RT1504

Figure 6.32: Hall ticket PDF



STATEMENT OF MARKS

End Semester Examination - 2026-27

RT SMS

Student Name: Rishabh Shah
Enrollment No: 23SS02IT001
Course: B.Sc. (IT)
Semester: 7
Exam Title: Semester 7 - Theory / Practical
Academic Year: 2026-27

Subject Name	Code	Internal /30	External /70	Total /100	Grade	Status
Image Processing	IT701	21	45	66	B	Pass
Devops	IT702	20	47	67	B	Pass
Cryptography and Network Security	IT703	18	45	63	B	Pass
Full Stack Development	IT704	19	36	55	C	Pass
ARVR	IT705	18	37	55	C	Pass

SGPA: 6.6 Total: 306 Credits: 20 Rank: 8 Result: PASS

Generated on 10/22/2026, 7:33:32 AM. This is a computer generated statement.

(c) 2026 RT SMS - Designed & Developed by RT1504

Figure 6.33: Result marksheet PDF

6.6 Mobile View

On small screens the sidebar becomes a slide-in menu and cards stack vertically. The automated tests confirmed that there is no sideways scrolling at 390 px width.

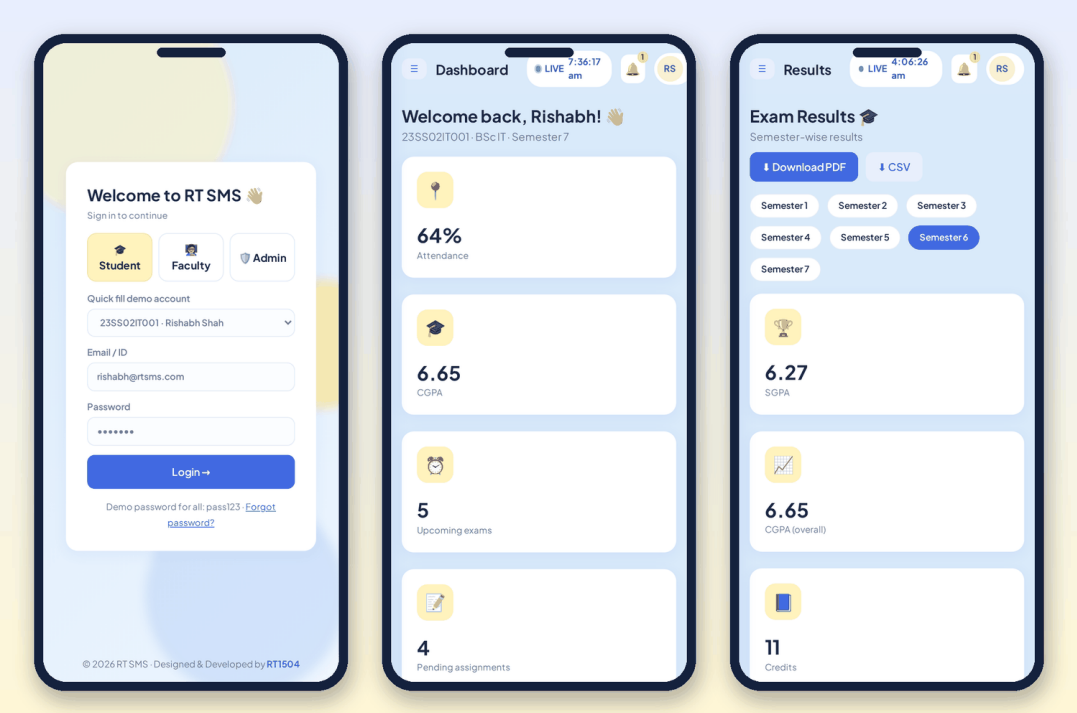


Figure 6.34: Mobile login, dashboard and results

CHAPTER 7

LEARNING OUTCOMES

7.1 Skills and Knowledge Gained

- Designing a multi-role web application with separate menus, permissions and workflows.
- Managing application state in one object and keeping it consistent across pages and tabs.
- Using LocalStorage, SessionStorage and BroadcastChannel for persistence and live synchronisation.
- Building reusable UI blocks (cards, tables, modals, toasts) with template literals.
- Implementing real rules: grade points, SGPA / CGPA, ranking, attendance percentage, fee calculation.
- Generating PDF documents and QR codes in the browser.
- Making a responsive layout and testing it at desktop, tablet and mobile sizes.
- Writing automated browser tests and publishing a project on GitHub Pages.

7.2 Challenges and Solutions

Challenge	Solution
Keeping Admin, Faculty and Student screens consistent without a server	One shared state object saved to LocalStorage and broadcast to other tabs with BroadcastChannel.
Live refresh must not erase what the user is typing	`sync()` re-renders only when no dialog is open and no input field is focused.
Creating many PDF layouts	Common helper functions `hdr()`, `kv()`, `pt()` and `ftr()` reused by every document.
Working without internet for libraries	The app runs fully offline; only PDF and QR need the CDN and show a message if missing.
Realistic demo data	A seed function generates 12 students, 8 faculty, marks, attendance and fees with a repeatable pattern.
Preventing access to the wrong portal	`boot(role)` checks session, role and status on every portal page load.

7.3 Limitations

- Data is stored in one browser; there is no central database or multi-device sharing.
- Passwords are kept in plain form in demo data; there is no server-side security.
- Payment is a demonstration gateway, not a real bank integration.
- No e-mail or SMS notification is sent outside the browser.

CHAPTER

8

CONCLUSION

8.1 Conclusion

RT SMS shows that a complete, multi-role management system can be built with plain HTML, CSS and JavaScript. The project connects Admin, Faculty and Student through one shared data model, so attendance, leave, fees, exam forms and results flow naturally from one role to another and appear live on every screen. It produces professional PDF documents, calculates SGPA and CGPA automatically and works on desktop and mobile.

The 35 automated tests, all of which passed, and the detailed workflow diagrams show that the system behaves as designed. The project has strengthened my understanding of state management, event-driven programming, workflow design and front-end testing, and it is a strong base for a full-stack version.

8.2 Future Enhancements

Enhancement	Benefit
Node.js / Express backend with MongoDB or MySQL	Central database, multi-device access and real users
Secure login: hashed passwords, JWT / sessions	Production-level security and role permissions on the server
Real payment gateway (Razorpay, UPI intent)	Genuine fee collection with verified receipts
E-mail, SMS and push notifications	Alerts for low attendance, results and notices
QR / biometric attendance	Faster and more reliable attendance marking
Parent portal and analytics dashboards	Wider access and better decision support
Progressive Web App and dark mode	Install on phone, offline use and better comfort

CHAPTER 9

REFERENCES AND GLOSSARY

9.1 References

1. RT SMS live project — https://shahrishabh1513-jsk.github.io/RT-SMS-Student-Manegment-System/
2. RT SMS source code (GitHub) — https://github.com/shahrishabh1513-jsk/RT-SMS-Student-Manegment-System
3. MDN Web Docs – HTML, CSS and JavaScript — https://developer.mozilla.org/
4. MDN – BroadcastChannel API — https://developer.mozilla.org/en-US/docs/Web/API/BroadcastChannel
5. MDN – Web Storage API — https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
6. jsPDF library — https://github.com/parallax/jsPDF
7. qrcode-generator library — https://github.com/kazuhikoarase/qrcode-generator
8. GitHub Pages documentation — https://docs.github.com/pages
9. Playwright browser testing — https://playwright.dev/

9.2 Glossary

Term	Meaning
SMS	Student Management System
SGPA / CGPA	Semester / Cumulative Grade Point Average
LocalStorage	Browser storage that keeps data after the tab is closed
SessionStorage	Browser storage valid for one tab session (used for login)
BroadcastChannel	Browser API that lets tabs of the same site send messages to each other
jsPDF	JavaScript library for creating PDF files
GitHub Pages	Free static website hosting from a GitHub repository
Role-based access	Different menus and permissions for Admin, Faculty and Student
UTR	Unique transaction reference number of a UPI payment

CONNECT WITH ME

Thank you for reading this report. Use the buttons below to explore the live project, read the source code or get in touch.

OPEN LIVE PROJECT

VIEW PROJECT REPOSITORY

GITHUB PROFILE

LINKEDIN PROFILE



SHAH RISHABH ALPESHBHAI

Full Stack Developer · RT1504

"Better Code, Bigger Dreams"